# Machine Learning Operations (MLOps) AIMLCZG523
# Assignment 1 - Heart Disease Risk Prediction

Name: Kaushika S
Student ID: 2025AE05956
GitHub repository: https://github.com/2025ae05956/heart-disease-mlops
Demo video: final_video_recordings.mp4

(https://drive.google.com/file/d/19ImrMSql9Ag9qQ2e4SnRYUP759TA1wsL/view?usp=sharing)

# Table of Contents

1. Introduction
2. System Architecture
3. Setup and Installation Instructions
4. Data Acquisition and Exploratory Data Analysis
   4.1 Data acquisition
   4.2 Cleaning and preprocessing
   4.3 Visual analysis
5. Feature Engineering and Model Development
   5.1 Feature preparation
   5.2 Models
   5.3 Evaluation method
   5.4 Hyper-parameter tuning
   5.5 Results
   5.6 Model selection
6. Experiment Tracking with MLflow
7. Model Packaging and Reproducibility
8. CI/CD Pipeline and Automated Testing
   8.1 Unit tests
   8.2 GitHub Actions workflow
   8.3 Results
9. Model Containerisation
   9.1 API
   9.2 Docker image
   9.3 Build and test
10. Production Deployment on Kubernetes
11. Monitoring and Logging
12. Challenges and Solutions
13. Production-Readiness
14. Deliverables
15. Conclusion and Future Work

# 1. Introduction

The aim of this assignment was to build a machine learning classifier that predicts the presence of heart disease from 13 routine clinical measurements, and to take it all the way to a reproducible, tested, containerised and monitored API, following MLOps best practices.

I covered the complete lifecycle:

- data acquisition, cleaning and exploratory data analysis (EDA) of the UCI Heart Disease (Cleveland) dataset,
- feature preparation and the comparison of two classifiers, Logistic Regression and Random Forest, using cross-validation and hyper-parameter tuning,
- experiment tracking with MLflow,
- packaging of the final model together with its preprocessing pipeline,
- a CI/CD pipeline in GitHub Actions with linting, unit tests and automated training,
- a FastAPI prediction service packaged as a Docker container,
- deployment on Kubernetes (Minikube) behind a LoadBalancer,
- request logging and monitoring with Prometheus and Grafana.

The selected model is a tuned Logistic Regression. It achieved a 5-fold cross-validated ROC-AUC of 0.908 and, on the held-out test set, an accuracy of 0.869, a recall of 0.929 and a ROC-AUC of 0.958.

# 2. System Architecture

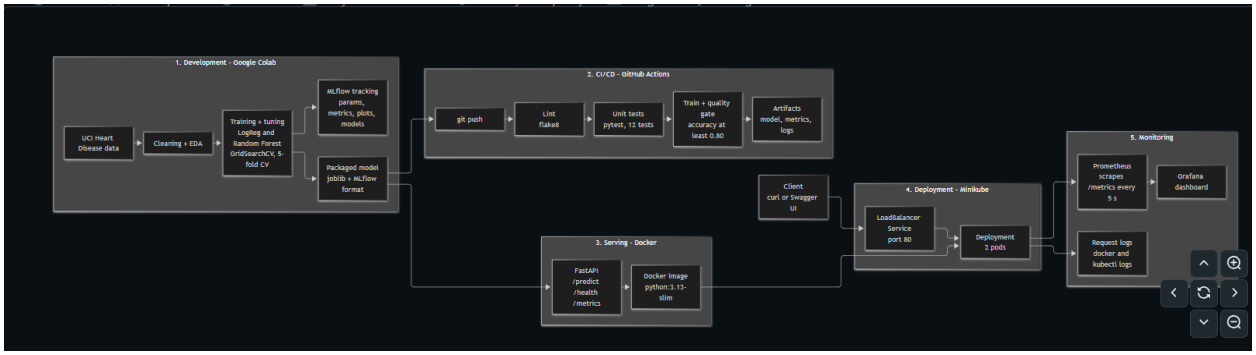



*Figure 1: System architecture*

The system is organised in five stages:

1. **Development (Google Colab):** I downloaded, cleaned and explored the data, trained and tuned the models, tracked every experiment with MLflow, and packaged the final pipeline.
2. **CI/CD (GitHub Actions):** every push to the `main` branch automatically runs three chained jobs (linting, unit tests, and model training with a quality gate) and stores their outputs as downloadable artifacts.
3. **Serving (Docker):** a FastAPI application exposes `/predict`, `/health` and `/metrics`. It is packaged in a Docker image based on `python:3.13-slim`, the same Python version used for training.
4. **Deployment (Kubernetes / Minikube):** the image runs as a Deployment with two replicas behind a LoadBalancer Service.
5. **Monitoring:** every request is logged, Prometheus collects the API's metrics every 5 seconds, and a Grafana dashboard visualises them.

**Tools used:** Python 3.13, pandas, NumPy, scikit-learn 1.6.1, matplotlib, seaborn, MLflow 3.17, pytest, flake8, GitHub Actions, FastAPI, Uvicorn, Docker, Docker Compose, Minikube, kubectl, Prometheus and Grafana. I used Google Colab for the notebooks and GitHub Codespaces (a cloud VS Code environment with Docker) for the container, Kubernetes and monitoring work.

---

# 3. Setup and Installation Instructions

**Prerequisites:** Python 3.13, Git and Docker; Minikube and kubectl for the Kubernetes deployment.

**Clone and install:**

```
git clone https://github.com/2025ae05956/heart-disease-mlops.git
cd heart-disease-mlops
python -m venv .venv
source .venv/bin/activate          (Windows: .venv\Scripts\activate)
pip install -r requirements.txt
```

**Run the ML pipeline:**

```
python scripts/download_data.py    # download and clean the dataset
python src/train.py                # train, evaluate, log to MLflow, save model
flake8 src tests                   # lint
pytest tests -v                    # unit tests
```

**Build and run the API container:**

```
docker build -t heart-api .
docker run -d -p 8000:8000 --name heart-api heart-api
```

curl -X POST http://localhost:8000/predict -H "Content-Type: application/json" -d @sample_input.json

**Deploy on Kubernetes (Minikube):**

```
minikube start --driver=docker --cni=bridge
minikube image load heart-api:latest
kubectl apply -f k8s/deployment.yaml -f k8s/service.yaml
minikube tunnel                  # in a second terminal
kubectl get svc heart-api        # note the EXTERNAL-IP
curl http://<EXTERNAL-IP>/health
```

**Start the monitoring stack:**

```
docker compose up -d             # API :8000, Prometheus :9090, Grafana :3000
python3 scripts/generate_traffic.py --n 200
```

**Notebooks:** the three notebooks in `notebooks/` run in Google Colab after uploading `data/heart_clean.csv` (Runtime → Run all).

**API access:** the API is not publicly hosted. For local testing it is available at `http://localhost:8000` (interactive documentation at `http://localhost:8000/docs`) when run with Docker, or at the LoadBalancer IP when deployed on Minikube.

---

# 4. Data Acquisition and Exploratory Data Analysis

## 4.1 Data acquisition

I used the **UCI Heart Disease dataset (Cleveland subset)** from the UCI Machine Learning Repository (https://archive.ics.uci.edu/dataset/45/heart+disease). It contains 303 patients, 13 clinical features and a target column `num`.

The data can be obtained in two reproducible ways:

- `scripts/download_data.py` downloads `processed.cleveland.data` from UCI, cleans it and writes `data/heart_clean.csv`;
- The first cell of `notebooks/01_EDA_Heart_Disease.ipynb` downloads the same file with `wget`.

Manual alternative: download the file from the UCI page and place it in `data/raw/`.

The 13 features are: age, sex, chest pain type (cp), resting blood pressure (trestbps), cholesterol (chol), fasting blood sugar (fbs), resting ECG (restecg), maximum heart rate (thalach), exercise-induced angina (exang), ST depression (oldpeak), slope of the ST segment (slope), number of major vessels (ca) and thallium test result (thal).

## 4.2 Cleaning and preprocessing

| Step | What I did | Why |
|---|---|---|
| Column names | assigned the 14 documented names | the raw file has no header |
| Missing values | ? read as missing: **4 in `ca`, 2 in `thal`** (6 of 3,939 values) | the UCI file marks missing values with ? |
| Imputation | filled with the column **mode** (most frequent value) | only 6 cells; deleting rows would lose patients, and both columns are categorical codes, so the mode is the natural choice |
| Target | `num` 0 → 0 (no disease), 1–4 → 1 (disease) | the problem asks for a binary target |
| Duplicates | checked and removed (none found) | avoid double-counting patients |
| Encoding | one-hot encoding of `cp`, `restecg`, `slope`, `thal` | these are category codes, not quantities (chest pain type 4 is not “twice” type 2) |

The cleaned dataset has 303 rows and 14 columns (13 features + target) with no missing values, and is saved as `data/heart_clean.csv`.

## 4.3 Visual analysis

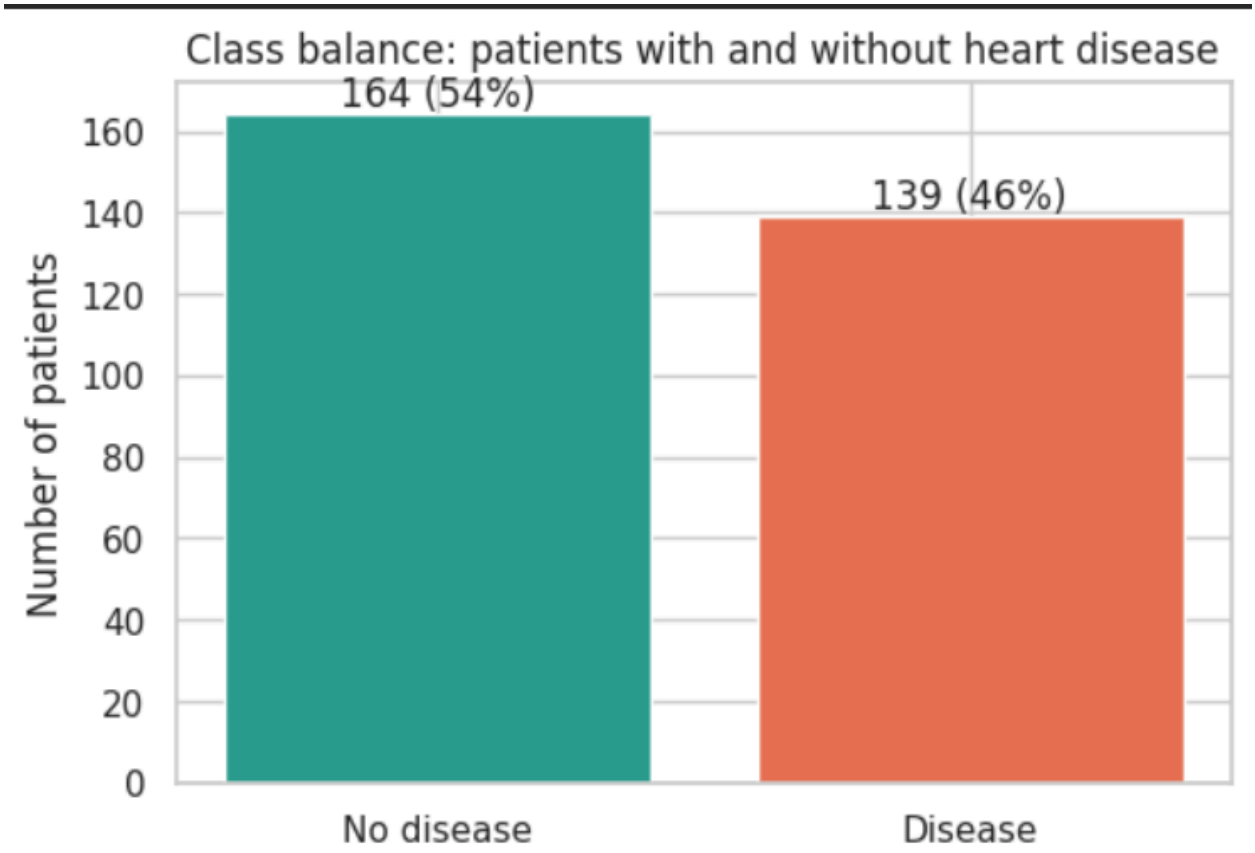



*Figure 2: Class balance*

The classes are fairly balanced: **164 patients without disease (54%)** and **139 with disease (46%)**. Accuracy is therefore a meaningful metric, but because missing a sick patient is costly in a medical setting, I also report precision, recall and ROC-AUC, and I used stratified splits everywhere so both classes keep the same ratio.

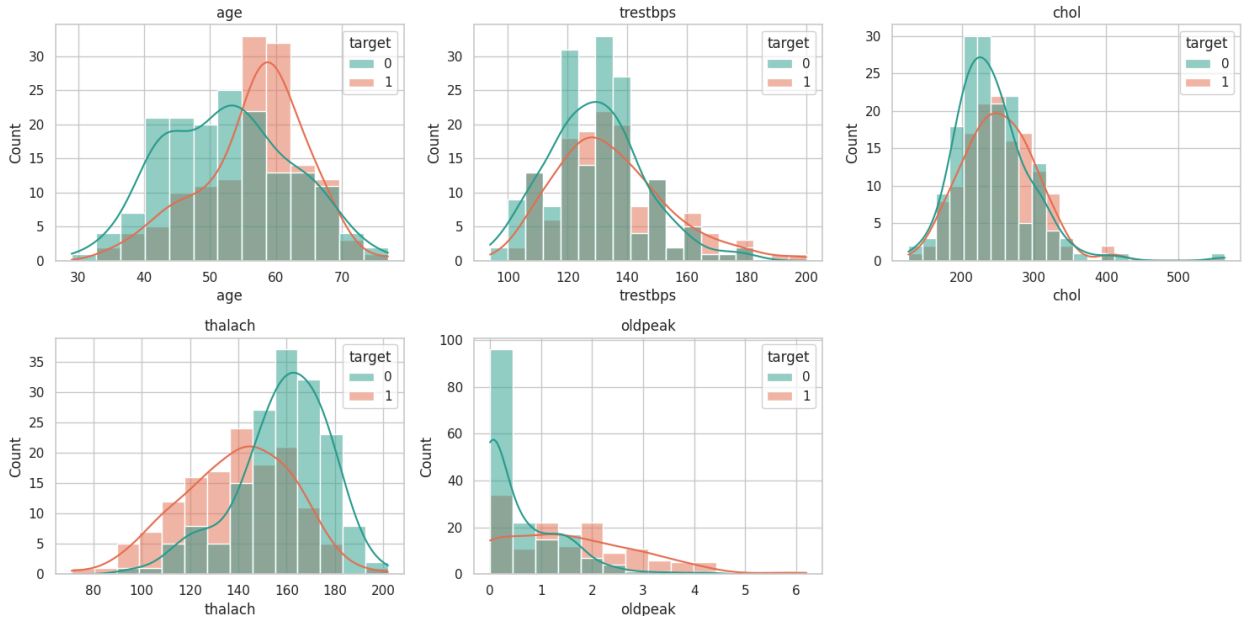



*Figure 3: Distributions of numeric features by diagnosis*

The histograms show that patients with heart disease reach a **lower maximum heart rate (thalach)** and have **higher ST depression (oldpeak)**. Age is slightly higher in the disease group, while cholesterol and resting blood pressure overlap strongly between the two groups. The numeric features have very different scales (cholesterol around 250, oldpeak around 1), so they need to be standardised before modelling.

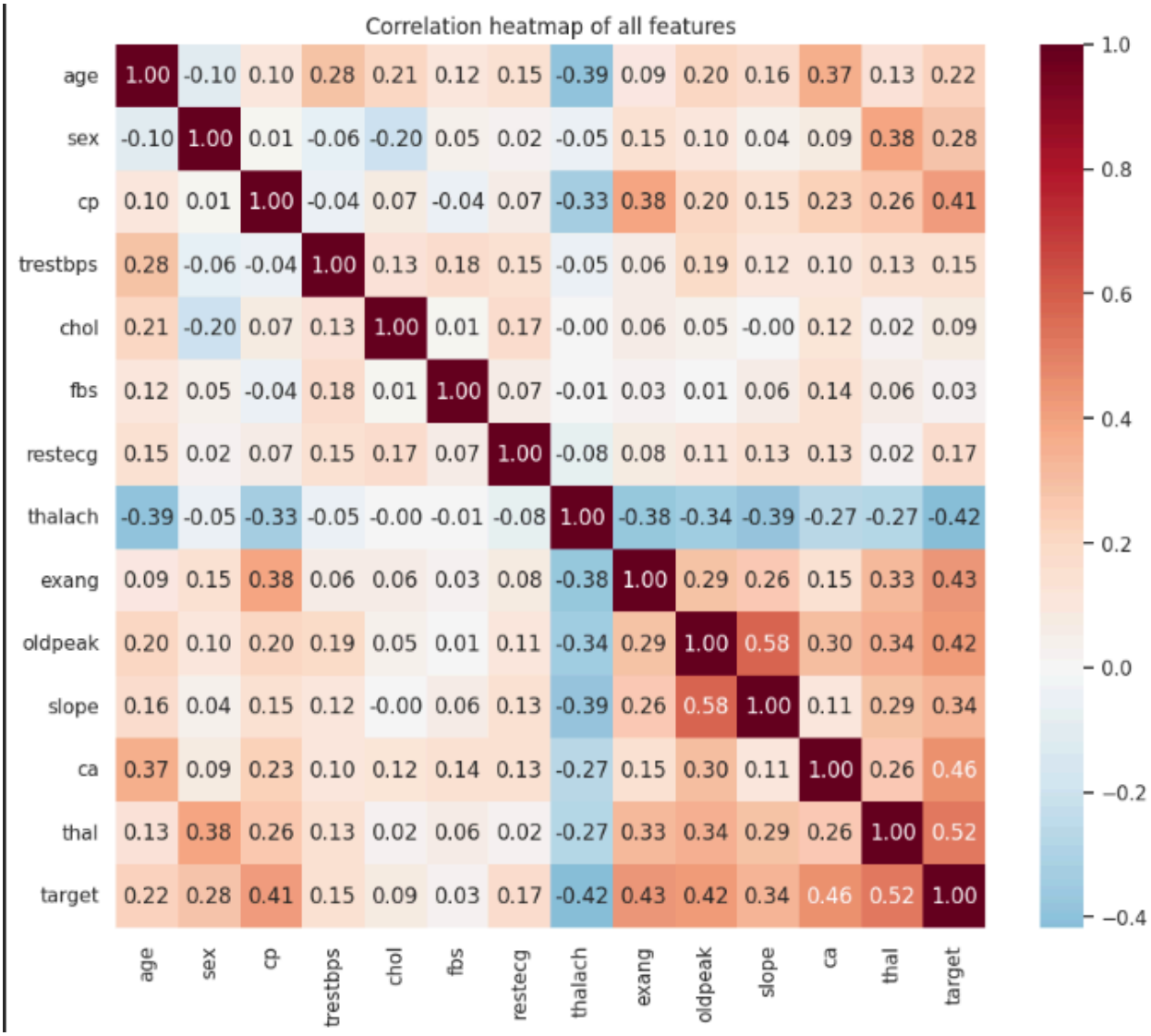



*Figure 4: Correlation heatmap*

The correlation with the target ranks the features as follows:

| Feature | Correlation with target |
|---|---|
| thal | 0.52 |
| ca | 0.46 |
| exang | 0.43 |
| oldpeak | 0.42 |
| cp | 0.41 |

| | |
|---|---|
| slope | 0.34 |
| sex | 0.28 |
| age | 0.22 |
| restecg | 0.17 |
| trestbps | 0.15 |
| chol | 0.09 |
| fbs | 0.03 |
| thalach | −0.42 |

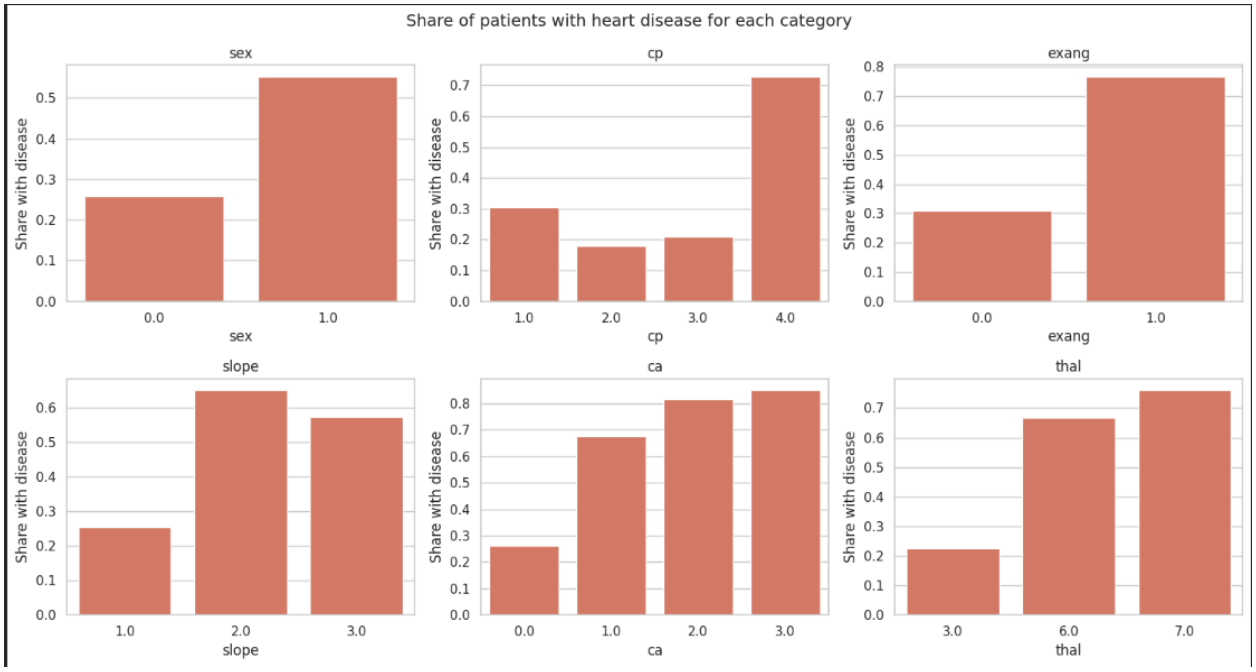



*Figure 5: Share of patients with disease per category*

**Key EDA findings:**

- The strongest signals are the thallium result (thal), the number of blocked vessels (ca), exercise-induced angina (exang), ST depression (oldpeak) and chest pain type (cp).
- Maximum heart rate is negatively correlated (−0.42): sick patients reach a lower peak heart rate during exercise.
- Surprisingly, cholesterol (0.09) and fasting blood sugar (0.03) are only weakly related to heart disease on their own in this dataset.
- Chest pain type has a non-linear relationship: asymptomatic chest pain (cp = 4) has the highest disease rate. This confirms that cp must be one-hot encoded rather than treated

as a number. Because thal and cp are codes, their correlation values are only approximate.
- No two features are so strongly correlated with each other that one has to be removed, so I kept all 13 features.

---

# 5. Feature Engineering and Model Development

## 5.1 Feature preparation

I split the data **80/20 with stratification** (`random_state = 42`): **242 patients for training and 61 for testing**. The test set was kept aside and used only for the final evaluation, never for tuning or model selection.

All preprocessing is done inside a scikit-learn `ColumnTransformer`, combined with the model in one `Pipeline`:

| Feature group | Columns | Transformation |
|---|---|---|
| Numeric | age, trestbps, chol, thalach, oldpeak, ca | `StandardScaler` (mean 0, standard deviation 1) |
| Binary | sex, fbs, exang | kept as 0/1 (passthrough) |
| Categorical | cp, restecg, slope, thal | `OneHotEncoder(handle_unknown="ignore")` |

Putting the preprocessing inside the pipeline has two advantages: during cross-validation the scaler is fitted only on the training folds (no data leakage), and the same transformation is saved with the model and reused automatically by the API. `handle_unknown="ignore"` keeps the model from crashing if a category value it has never seen arrives in production.

## 5.2 Models

I trained the two models asked for:

- **Logistic Regression:** a simple, interpretable linear model whose coefficients show how each feature raises or lowers the risk;
- **Random Forest:** an ensemble of decision trees that can capture non-linear relationships and feature interactions.

## 5.3 Evaluation method

All models were evaluated with **stratified 5-fold cross-validation** on the training set: the training data is split into 5 parts, the model is trained on 4 and tested on the 5th, five times, and the scores are averaged. I used **accuracy, precision, recall and ROC-AUC**. ROC-AUC measures how well the model ranks sick patients above healthy ones regardless of the 0.5 threshold, so I used it as the main metric for tuning and selection.

## 5.4 Hyper-parameter tuning

I tuned both models with `GridSearchCV` (stratified 5-fold CV, refit on ROC-AUC):

| Model | Search space | Combinations | Best parameters |
|---|---|---|---|
| Logistic Regression | C ∈ {0.01, 0.1, 1, 10}; class_weight ∈ {None, balanced} | 8 | C = 1, class_weight = balanced |
| Random Forest | n_estimators ∈ {100, 300}; max_depth ∈ {None, 4, 8}; min_samples_leaf ∈ {1, 3} | 12 | n_estimators = 300, max_depth = 4, min_samples_leaf = 3 |

For Logistic Regression, `C` controls regularisation, and `class_weight = balanced` gives slightly more weight to the smaller disease class, which improved recall. For Random Forest, the search chose **shallow trees** (depth 4, at least 3 patients per leaf); with only 242 training patients, simpler trees generalise better than deep trees that memorise individual patients.

## 5.5 Results

**Cross-validation results (training set, mean of 5 folds):**

| Model | Stage | Accuracy | Precision | Recall | ROC-AUC |
|---|---|---|---|---|---|
| Logistic Regression | Baseline | 0.839 | 0.877 | 0.766 | 0.907 |
| Logistic Regression | Tuned | **0.859** | 0.870 | **0.828** | **0.908** |
| Random Forest | Baseline | 0.806 | 0.822 | 0.747 | 0.888 |
| Random Forest | Tuned | 0.826 | 0.850 | 0.764 | 0.898 |

**Test set results (61 unseen patients):**

| Model | Accuracy | Precision | Recall | ROC-AUC |
|---|---|---|---|---|
| Logistic Regression (tuned) | 0.869 | 0.812 | 0.929 | 0.958 |
| Random Forest (tuned) | 0.902 | 0.867 | 0.929 | 0.958 |

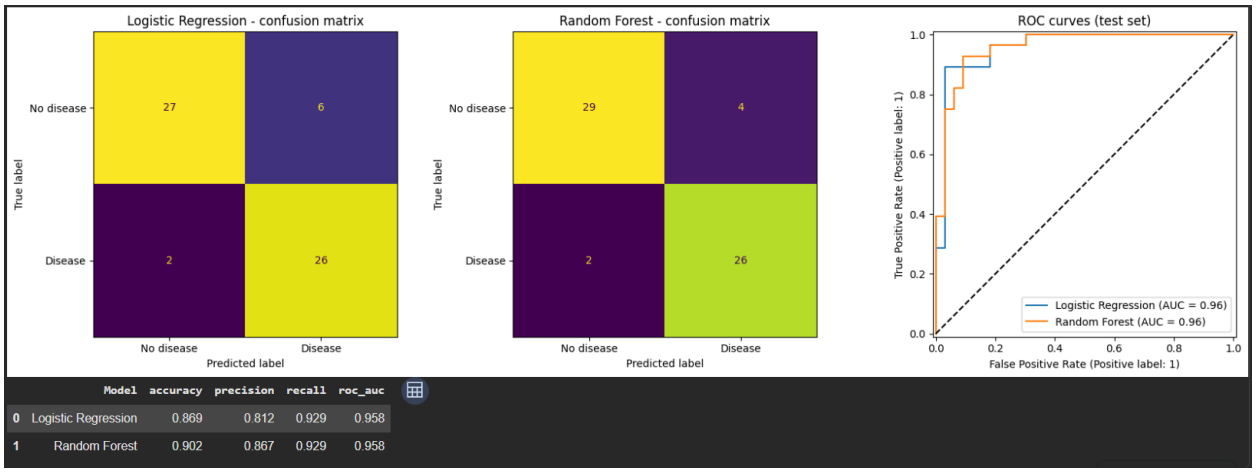



*Figure 6: Confusion matrices and ROC curves on the test set*

Tuning improved both models; for example, Logistic Regression recall rose from 0.766 to 0.828.

### 5.6 Model selection

I selected **Logistic Regression (C = 1, class_weight = balanced)**:

- It had the best **cross-validated** scores on every important metric (ROC-AUC 0.908, accuracy 0.859, recall 0.828), compared with Random Forest (0.898, 0.826, 0.764). Cross-validation averages five evaluations over 242 patients, so it is a more reliable basis for choosing than a single test set.
- On the 61-patient test set Random Forest had slightly higher accuracy (0.902 vs 0.869), but the confusion matrices show this is a difference of only **2 false alarms** (6 vs 4). Both models found **26 of the 28 sick patients** (recall 0.929) and had the same ROC-AUC (0.958). I treated this small difference as noise rather than a reason to change the choice, because choosing a model on the test set would bias the final evaluation.
- Logistic Regression is simpler, faster and **interpretable**, which matters in a medical setting where doctors need to understand why a prediction was made.

---

## 6. Experiment Tracking with MLflow

I integrated **MLflow 3.17** into the training notebook (`notebooks/02_Model_Training.ipynb`). The tracking store is a **SQLite database** (`heart_mlflow.db`), the recommended backend in current MLflow versions, and artifacts are stored in the local `mlruns/` folder. All runs belong to the experiment **heart-disease-classification**.

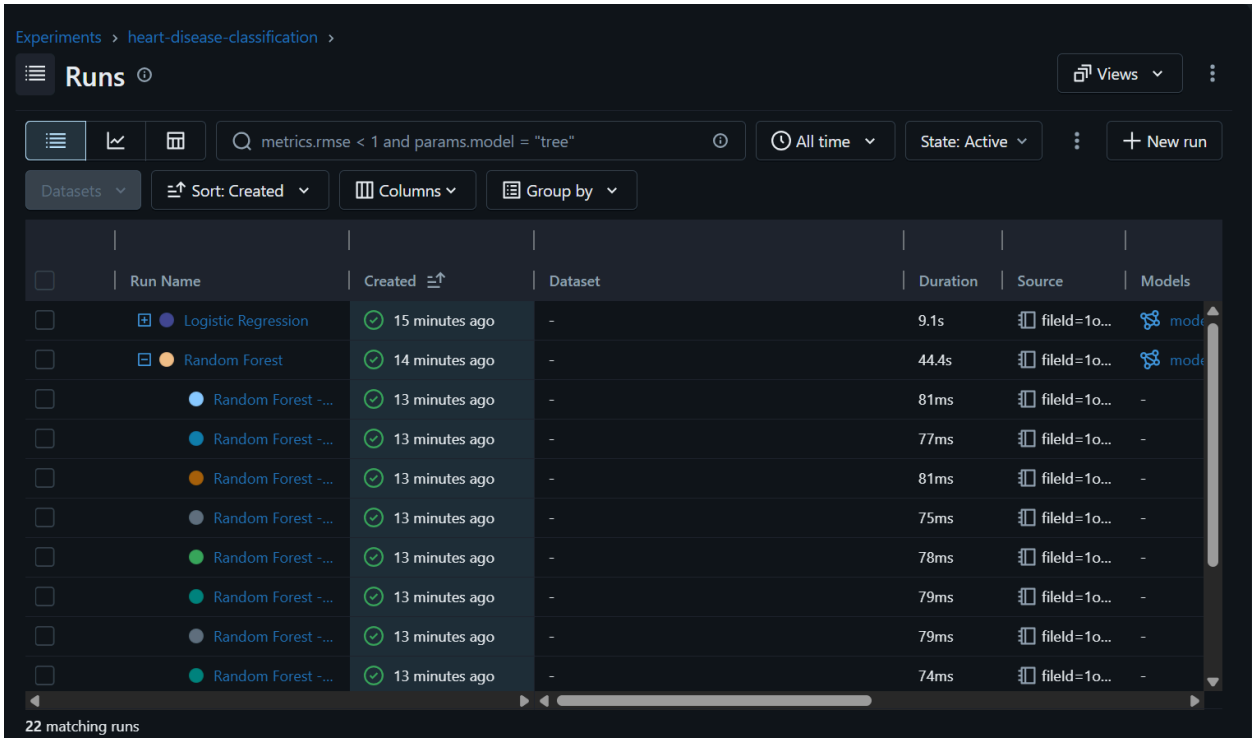
**Run structure:**

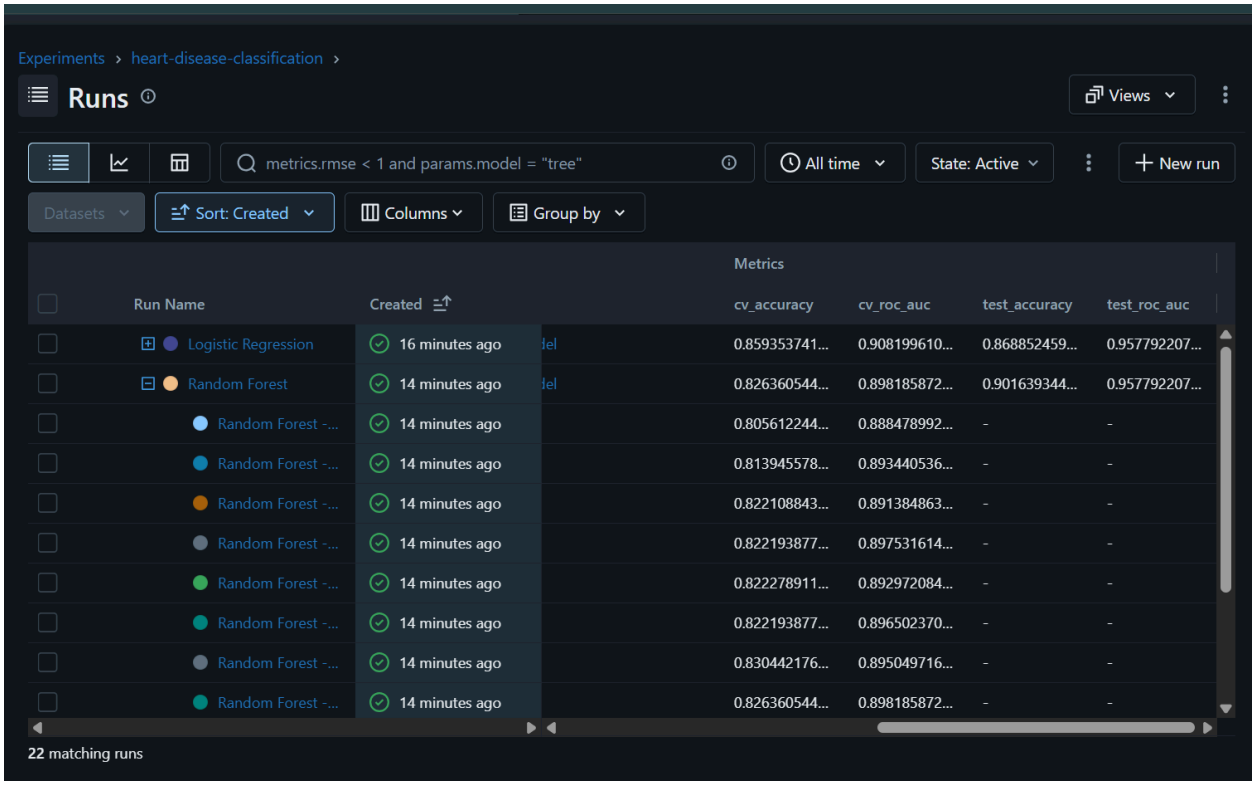
- one **parent run per model** (Logistic Regression and Random Forest),
- one **nested child run for every hyper-parameter combination** tried by GridSearchCV (8 for Logistic Regression, 12 for Random Forest).

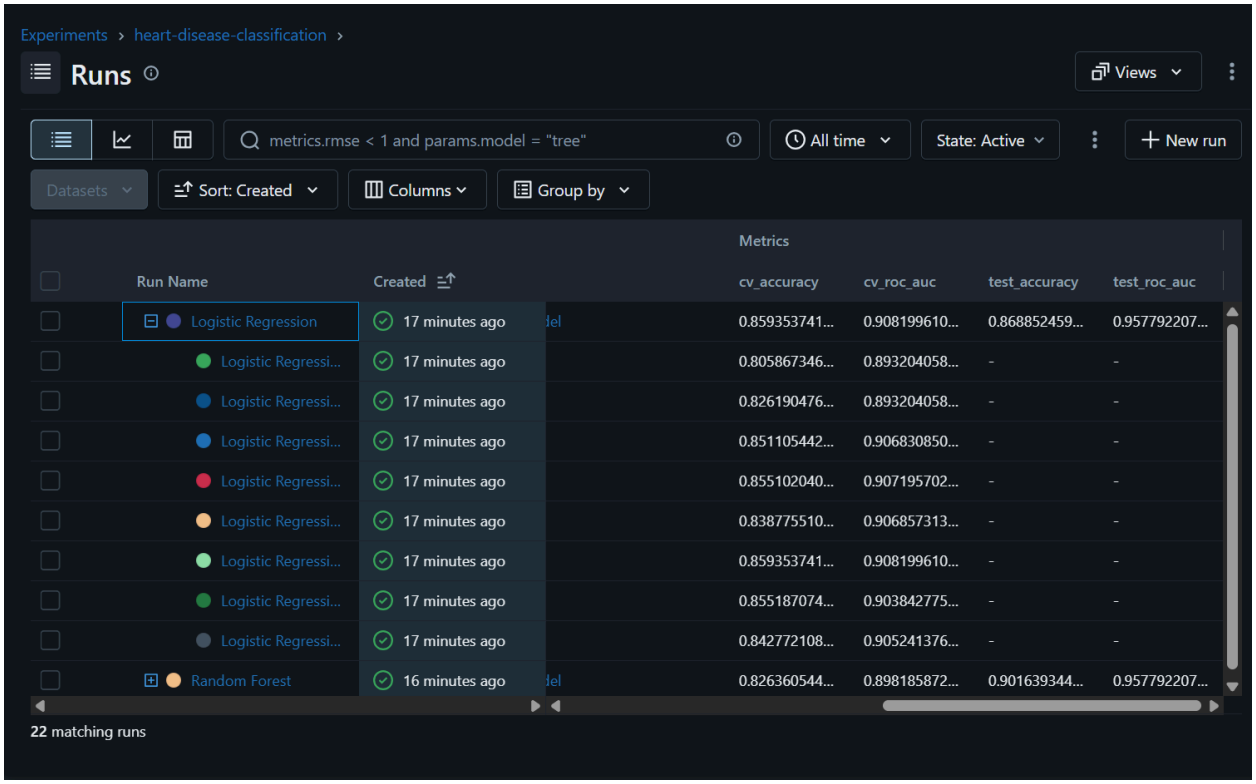
In total **22 runs** were tracked (2 model runs + 20 tuning trials).

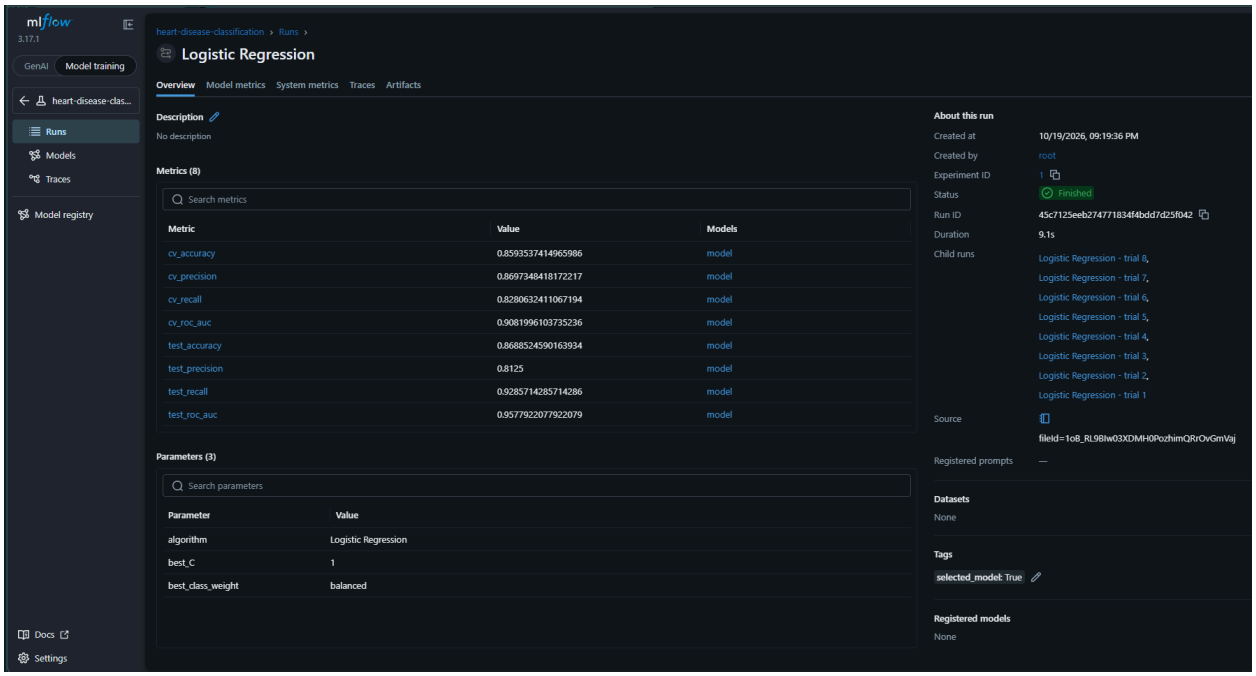
**What is logged:**

| Type | Content |
|---|---|
| Parameters | algorithm name; the parameters of every trial; the best parameters of each model |
| Metrics | mean 5-fold CV accuracy, precision, recall and ROC-AUC (every trial and the best model); test accuracy, precision, recall and ROC-AUC |
| Plots | `evaluation_plots.png`: confusion matrix and ROC curve for each model |
| Artifacts | the trained pipeline in MLflow model format, with its input signature and an input example |
| Tags | `selected_model` marks the chosen model |


Experiments › heart-disease-classification ›
Runs
Views
metrics.rmse < 1 and params.model = "tree"
All time
State: Active
New run
Datasets
Sort: Created
Columns
Group by
Run Name
Created
Dataset
Duration
Source
Models
Logistic Regression
15 minutes ago
9.1s
Random Forest
14 minutes ago
44.4s
Random Forest -...
13 minutes ago
81ms
77ms
81ms
75ms
78ms
79ms
79ms
74ms
22 matching runs


Experiments › heart-disease-classification ›
Runs
Views
metrics.rmse < 1 and params.model = "tree"
All time
State: Active
New run
Datasets
Sort: Created
Columns
Group by
Metrics
Run Name
Created
cv_accuracy
cv_roc_auc
test_accuracy
test_roc_auc
Logistic Regression
16 minutes ago
0.859353741...
0.908199610...
0.868852459...
0.957792207...
Random Forest
14 minutes ago
0.826360544...
0.898185872...
0.901639344...
0.957792207...
Random Forest -...
0.805612244...
0.888478992...
0.813945578...
0.893440536...
0.822108843...
0.891384863...
0.822193877...
0.897531614...
0.822278911...
0.892972084...
0.822193877...
0.896502370...
0.830442176...
0.895049716...
0.826360544...
0.898185872...
22 matching runs


Experiments › heart-disease-classification ›
Runs
Views
metrics.rmse < 1 and params.model = "tree"
All time
State: Active
New run
Datasets
Sort: Created
Columns
Group by
Metrics
Run Name
Created
cv_accuracy
cv_roc_auc
test_accuracy
test_roc_auc
Logistic Regression
17 minutes ago
0.859353741...
0.908199610...
0.868852459...
0.957792207...
Logistic Regressi...
17 minutes ago
0.805867346...
0.893204058...
Logistic Regressi...
17 minutes ago
0.826190476...
0.893204058...
Logistic Regressi...
17 minutes ago
0.851105442...
0.906830850...
Logistic Regressi...
17 minutes ago
0.855102040...
0.907195702...
Logistic Regressi...
17 minutes ago
0.838775510...
0.906857313...
Logistic Regressi...
17 minutes ago
0.859353741...
0.908199610...
Logistic Regressi...
17 minutes ago
0.855187074...
0.903842775...
Logistic Regressi...
17 minutes ago
0.842772108...
0.905241376...
Random Forest
16 minutes ago
0.826360544...
0.898185872...
0.901639344...
0.957792207...
22 matching runs


mlflow
3.17.1
GenAI
Model training
heart-disease-clas...
Runs
Models
Traces
Model registry
Docs
Settings
heart-disease-classification › Runs ›
Logistic Regression
Overview
Model metrics
System metrics
Traces
Artifacts
Description
No description
Metrics (8)
Search metrics
Metric
Value
Models
cv_accuracy 0.8593537414965986 model
cv_precision 0.8697348418172217 model
cv_recall 0.8280632411067194 model
cv_roc_auc 0.9081996103735236 model
test_accuracy 0.8688524590163934 model
test_precision 0.8125 model
test_recall 0.9285714285714286 model
test_roc_auc 0.9577922077922079 model
Parameters (3)
Search parameters
Parameter
Value
algorithm Logistic Regression
best_C 1
best_class_weight balanced
About this run
Created at 10/19/2026, 09:19:36 PM
Created by root
Experiment ID 1
Status Finished
Run ID 45c7125eeb274771834f4bdd7d25f042
Duration 9.1s
Child runs
Logistic Regression - trial 8,
Logistic Regression - trial 7,
Logistic Regression - trial 6,
Logistic Regression - trial 5,
Logistic Regression - trial 4,
Logistic Regression - trial 3,
Logistic Regression - trial 2,
Logistic Regression - trial 1
Source
fileId=1oB_RL9Blw03XDMH0PozhimQjRrOvGmVaj
Registered prompts —
Datasets
None
Tags
selected_model: True
Registered models
None

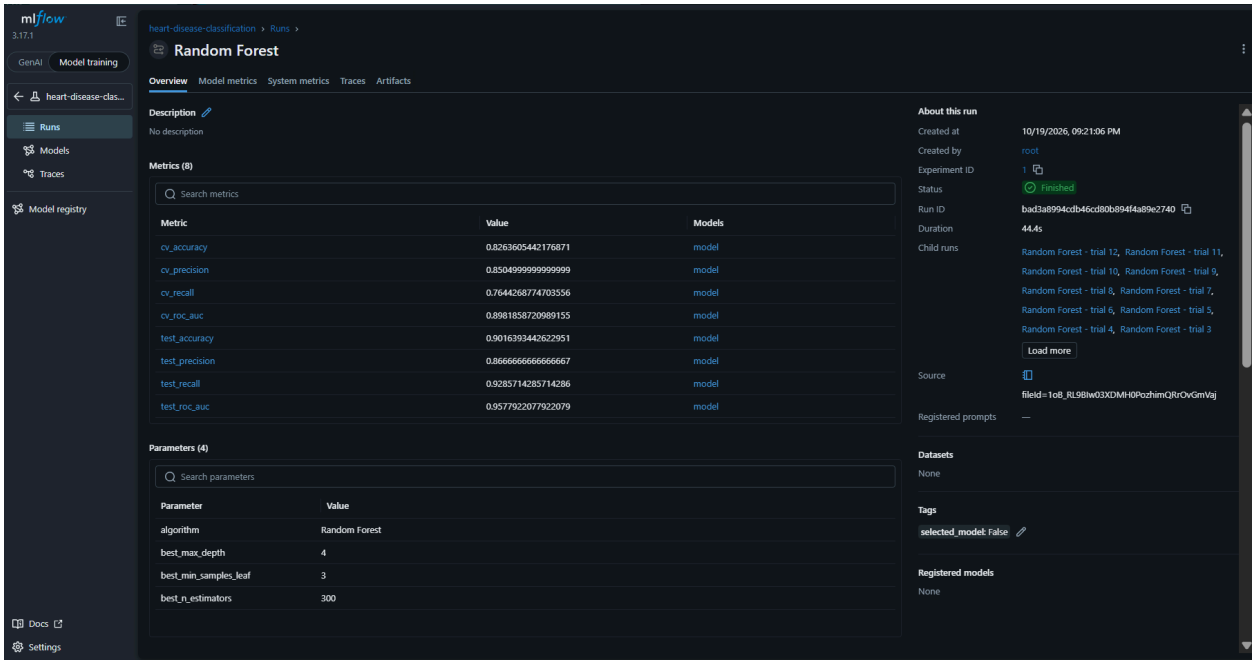



*Figure 7: MLflow experiment with the model runs and their tuning trials*

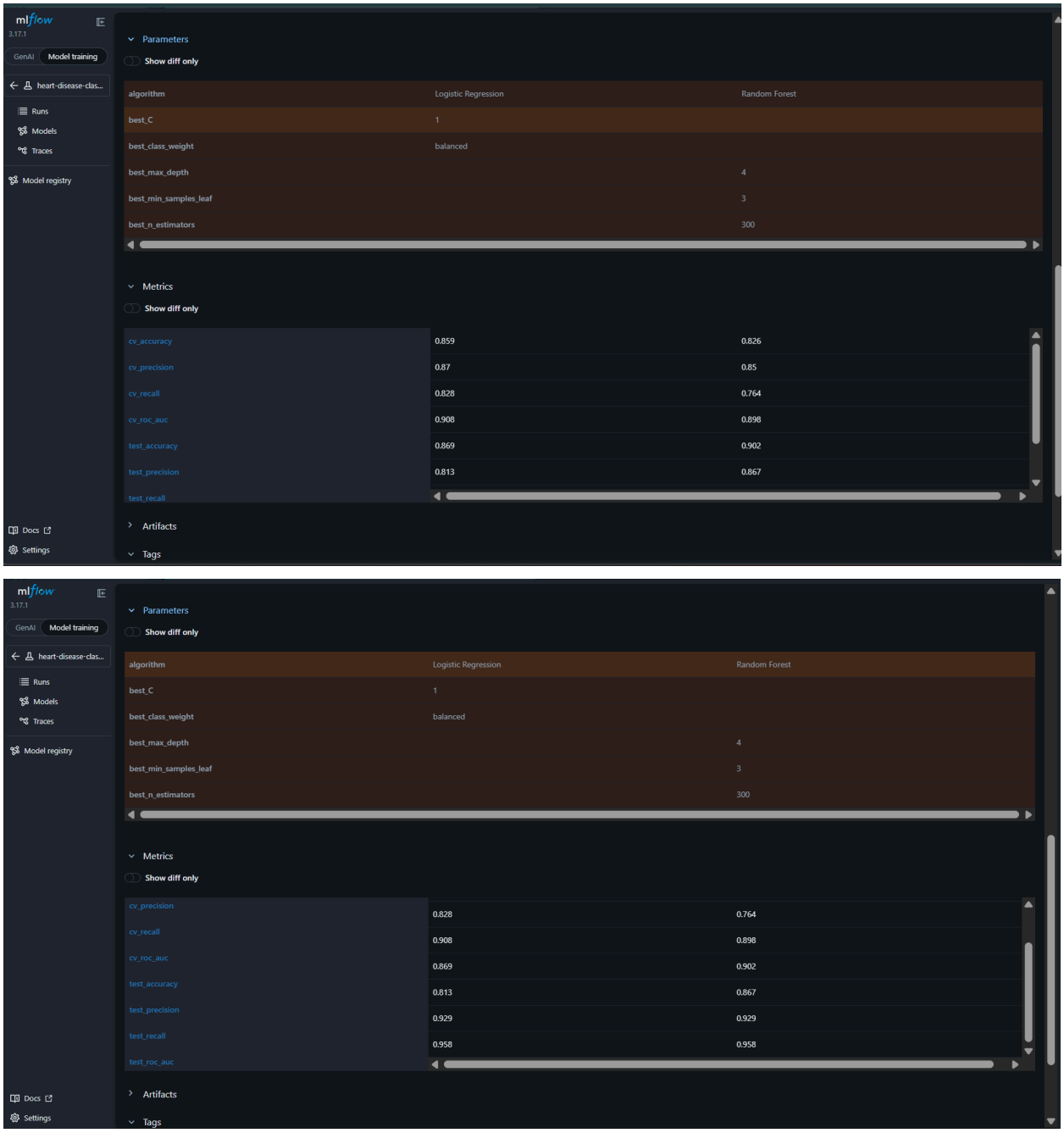



*Figure 8: Comparison of the two models in MLflow*

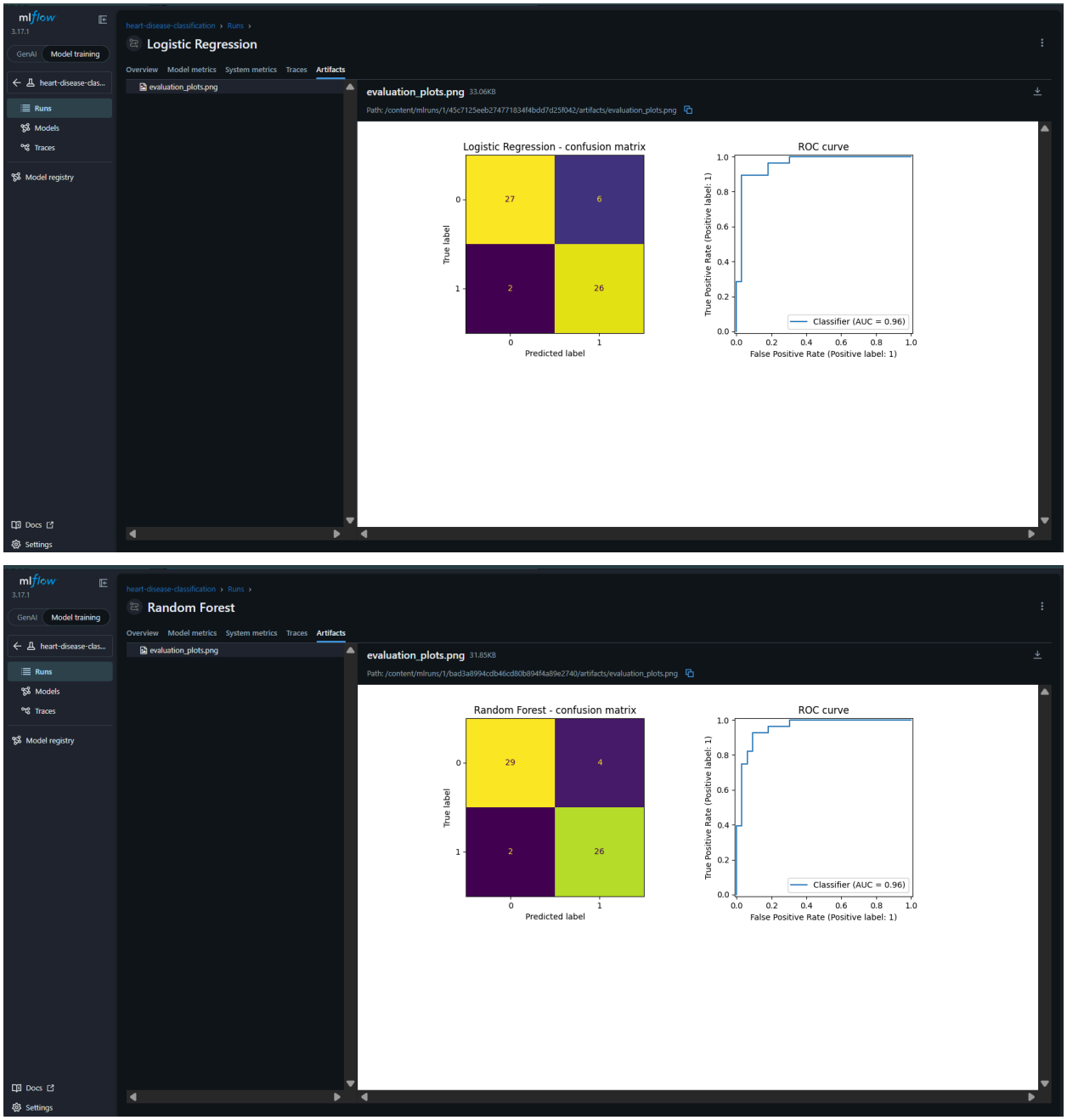



*Figure 9: Logged artifacts (evaluation plots and model) of the Logistic Regression run*

**Summary:** the tracked runs confirm the results in Section 5: the best Logistic Regression run has a CV ROC-AUC of 0.908 against 0.898 for the best Random Forest run, and all 20 trials can be compared in the MLflow UI to see how each hyper-parameter affected the scores. The training job in GitHub Actions (Section 8) also logs every CI training run to MLflow, in the experiment `heart-disease-ci`.

---

# 7. Model Packaging and Reproducibility

**Reusable preprocessing pipeline:** I moved the cleaning and preprocessing code out of the notebooks into one Python module, `src/heart_pipeline.py`, with three functions:

- `clean_raw()`: reads the raw UCI file and applies the cleaning steps of Section 4.2,
- `build_preprocessor()`: the `ColumnTransformer` of Section 5.1,
- `build_final_model()`: the preprocessor plus the selected Logistic Regression.

The training script, the unit tests and the notebooks all import this one file, so the data is always prepared exactly the same way.

**Saved model formats:**

- `heart_model.joblib`: the complete fitted pipeline (preprocessing + model) in one file; this is what the API loads;
- MLflow model folder: the same pipeline in MLflow format, including the required Python packages and the input signature;
- `model_metadata.json`: model name, parameters, feature list, test accuracy, random seed, scikit-learn version and training date.

MLflow 3 saves scikit-learn models with the secure **skops** format, which only loads explicitly trusted types. I whitelisted the scikit-learn column transformer (and the tree class for Random Forest), because these models were trained by me.

**Reproducibility check:** I reloaded the model from both formats and also retrained it from `heart_pipeline.py`, then compared the predicted probabilities on the test set:

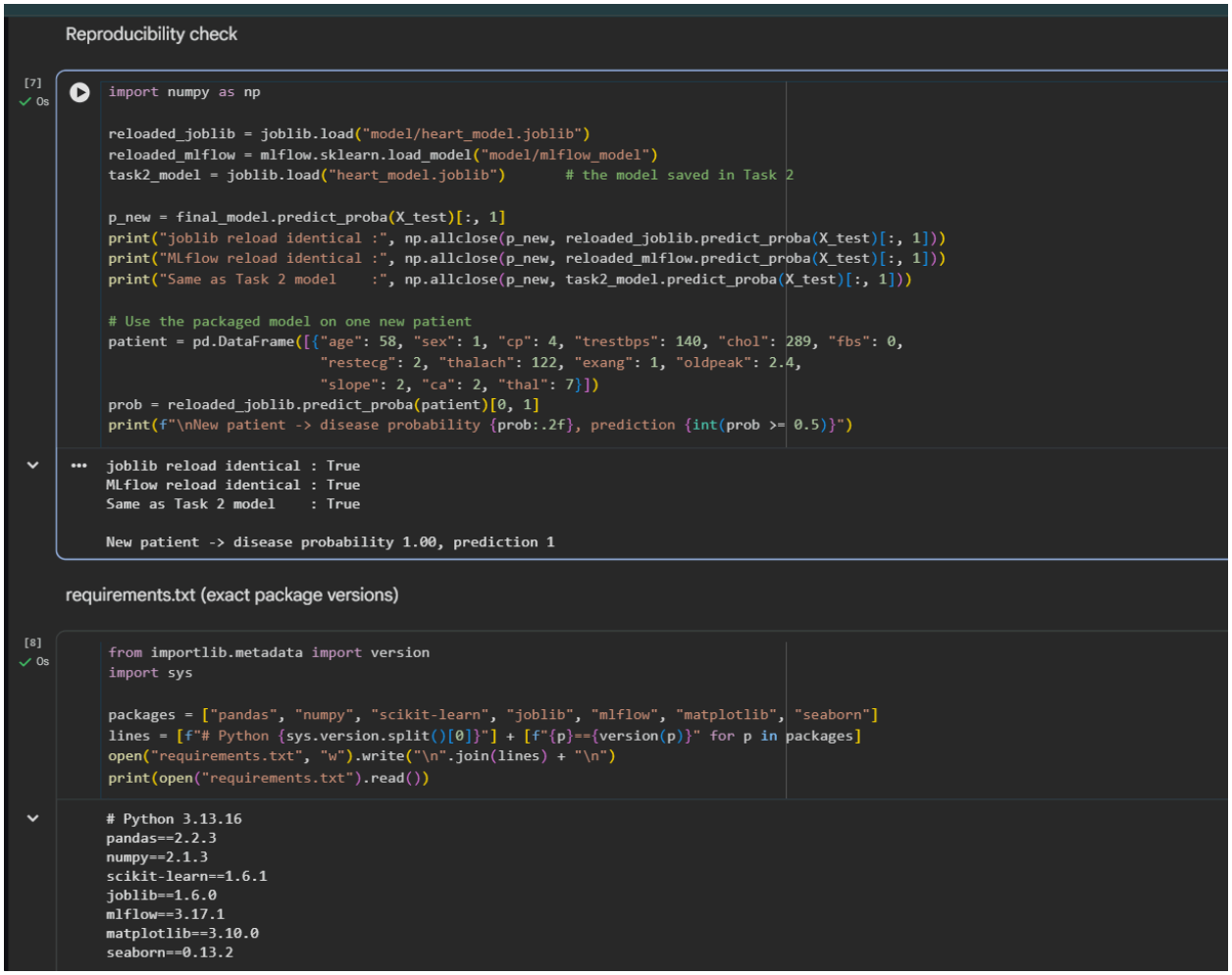



*Figure 10: Reproducibility check*

All three checks returned **True**: the joblib copy, the MLflow copy and the retrained model give identical predictions to the model from Section 5. A new patient given as raw values was scored correctly (probability 1.00, prediction 1), which shows the pipeline performs the scaling and encoding by itself.

**Environment:** `requirements.txt` pins the exact versions I used (Python 3.13, pandas 2.2.3, numpy 2.1.3, scikit-learn 1.6.1, joblib 1.6.0, mlflow 3.17.1, matplotlib 3.10.0, seaborn 0.13.2, pytest 8.3.4, flake8 7.1.1). A smaller `requirements-api.txt` contains only what the API needs. Pinning matters because a model saved with one scikit-learn version may not load correctly in another. A fixed `random_state = 42` is used for the split, the cross-validation and the models.

---

# 8. CI/CD Pipeline and Automated Testing

## 8.1 Unit tests

`tests/test_pipeline.py` contains **12 pytest unit tests**.

**Data processing (5 tests):** ? values are filled, the 0–4 target becomes 0/1, duplicate rows are removed, no feature columns are lost, and the real cleaned dataset is valid (more than 250 rows, no missing values, binary target). The data-cleaning tests use a small sample in the raw UCI format, including missing values and a duplicate row, so the expected result is known exactly.

**Model and pipeline (7 tests):** preprocessing output has no missing values, numeric features are standardised (mean 0), predicted probabilities are between 0 and 1, accuracy is above a 0.75 baseline, training twice gives identical predictions, an unseen category value does not crash the model, and a saved and reloaded model gives identical predictions.

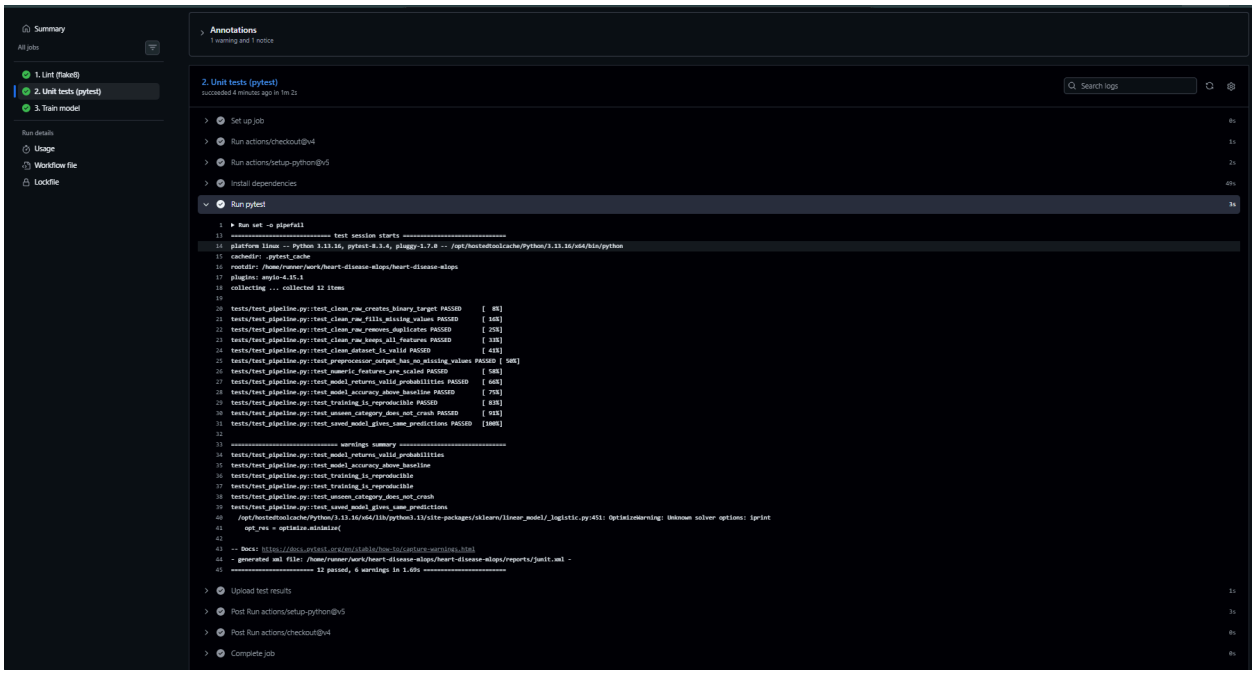



*Figure 11: All 12 unit tests passing in GitHub Actions*

## 8.2 GitHub Actions workflow

The workflow `.github/workflows/ci.yml` runs automatically on every push and pull request to `main`, and can also be started manually. It has three chained jobs:

| Job | What it does | Artifact uploaded |
|---|---|---|
| 1. Lint | runs `flake8` on `src/` and `tests/` (settings in `.flake8`) | `lint-report` |

| | | |
|---|---|---|
| 2. Unit tests | installs `requirements.txt`, runs `pytest` with a JUnit XML report | `test-results` |
| 3. Train model | runs `src/train.py`: 5-fold CV, test evaluation, saves model, metrics and plots, logs to MLflow, applies a **quality gate** (test accuracy ≥ 0.80); writes the metrics to the run summary | `trained-model` (model, metrics, plots, MLflow records, training log) |

Each job depends on the previous one (`needs:`), so if linting or a test fails, the later jobs do not run and no model is produced from broken code. Every command uses `set -o pipefail` with `tee`, so its output is saved to a log file and an error still fails the job. Each job uploads its logs and results as artifacts, so every workflow run is fully documented.

## 8.3 Results

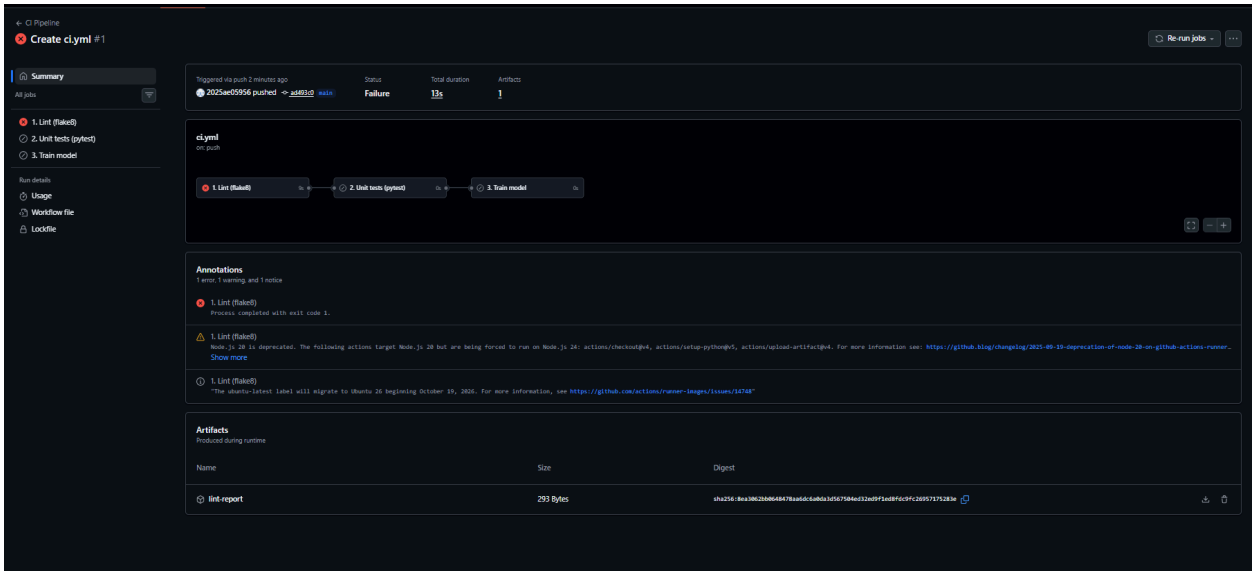



*Figure 12: First run failing at the lint stage*

My first pipeline run **failed at the lint stage**: flake8 reported unused imports (F401) and an unused variable (F841) in `src/train.py`, because the file had been only partly pasted. The test and training jobs were skipped automatically. This showed the pipeline fails on code errors and gives a clear log of the cause. After I fixed the file, the whole pipeline passed in about 2 minutes.

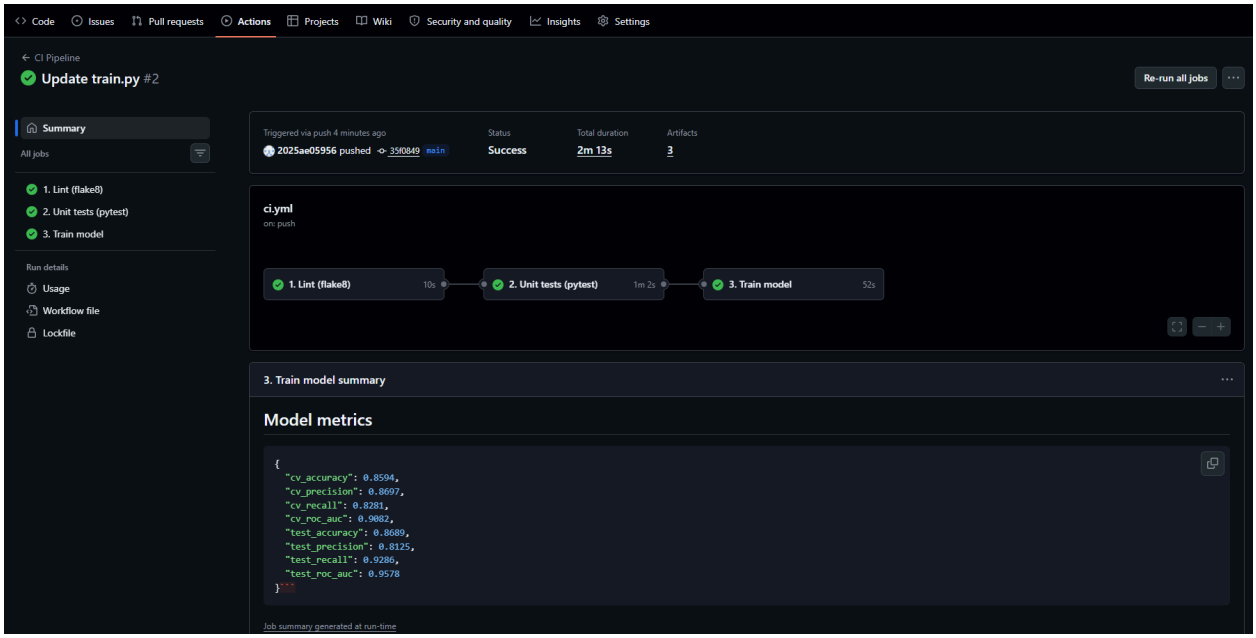



*Figure 13: Successful run of all three jobs and Model metrics published by the training job*

The CI training job reproduced the notebook results exactly: CV ROC-AUC 0.908, test accuracy 0.869, recall 0.929, ROC-AUC 0.958. This confirms the model can be rebuilt from a clean environment using only `requirements.txt`.




*Figure 15: Artifacts stored for each workflow run*

---

# 9. Model Containerisation

## 9.1 API

The prediction service (`api/main.py`) is built with **FastAPI**:

| Endpoint | Purpose |
|---|---|

| | |
|---|---|
| `GET /` | short welcome message |
| `GET /health` | health check: service up and model loaded |
| `POST /predict` | accepts one patient as JSON, returns prediction and confidence |
| `GET /metrics` | Prometheus metrics (Section 11) |
| `GET /docs` | interactive Swagger documentation (generated by FastAPI) |

The input is validated with a **Pydantic** model: every field is required and must be within a plausible range (for example age 1–120, chest pain type 1–4). Invalid input is rejected with **HTTP 422** and a message listing the wrong fields, instead of returning a meaningless prediction.

Example input (`sample_input.json`):

{"age": 58, "sex": 1, "cp": 4, "trestbps": 140, "chol": 289, "fbs": 0, "restecg": 2,
"thalach": 122, "exang": 1, "oldpeak": 2.4, "slope": 2, "ca": 2, "thal": 7}

Example response:

{"prediction": 1, "label": "heart disease", "probability_of_disease": 0.9972, "confidence": 0.9972}

`confidence` is the probability of the predicted class (the disease probability when the prediction is 1, and 1 minus it when the prediction is 0).

## 9.2 Docker image

The `Dockerfile` starts from **`python:3.13-slim`** (the Python version used for training), installs the pinned packages from `requirements-api.txt`, copies the API code and `model/heart_model.joblib`, and starts Uvicorn on port 8000. Packages are installed before the code is copied, so a code change rebuilds quickly from Docker's cache. The image contains only what is needed for serving (no MLflow, Jupyter or plotting libraries).

## 9.3 Build and test

I built and ran the container in GitHub Codespaces:

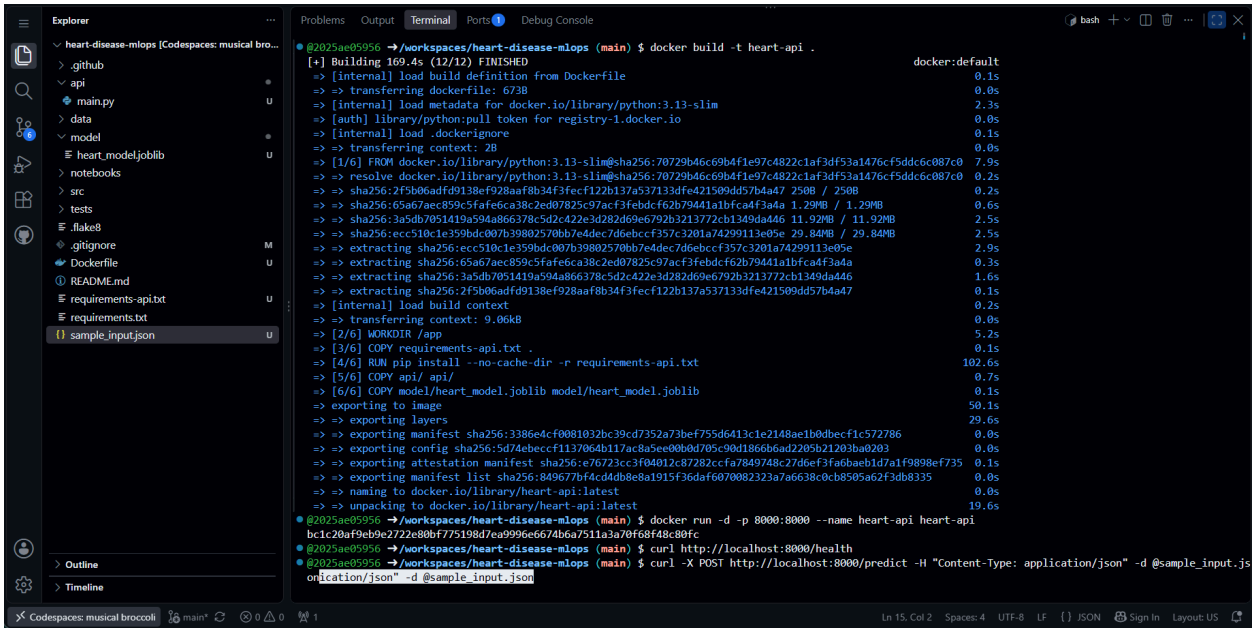



*Figure 16: Docker image build*

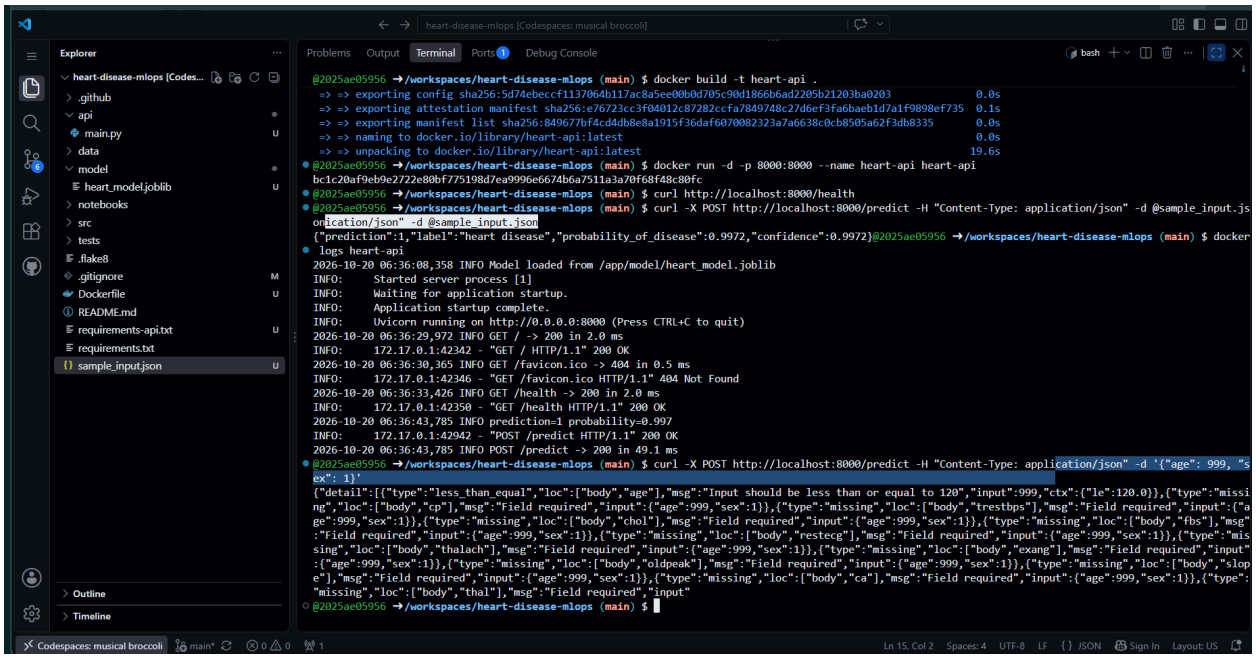



*Figure 17: Container running and answering /health and /predict with the sample input*

The container loaded the model, answered `GET /health` with `{"status":"ok","model_loaded":true}`, and returned **prediction 1 with confidence 0.997** for the sample patient in about 49 ms.

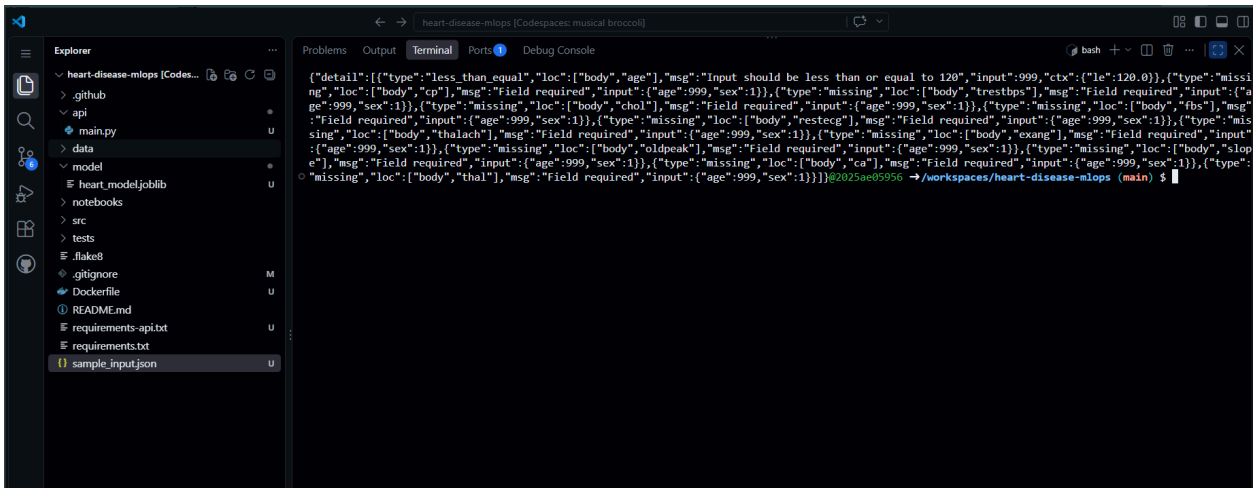



*Figure 18: Invalid input (age = 999) rejected with HTTP 422*

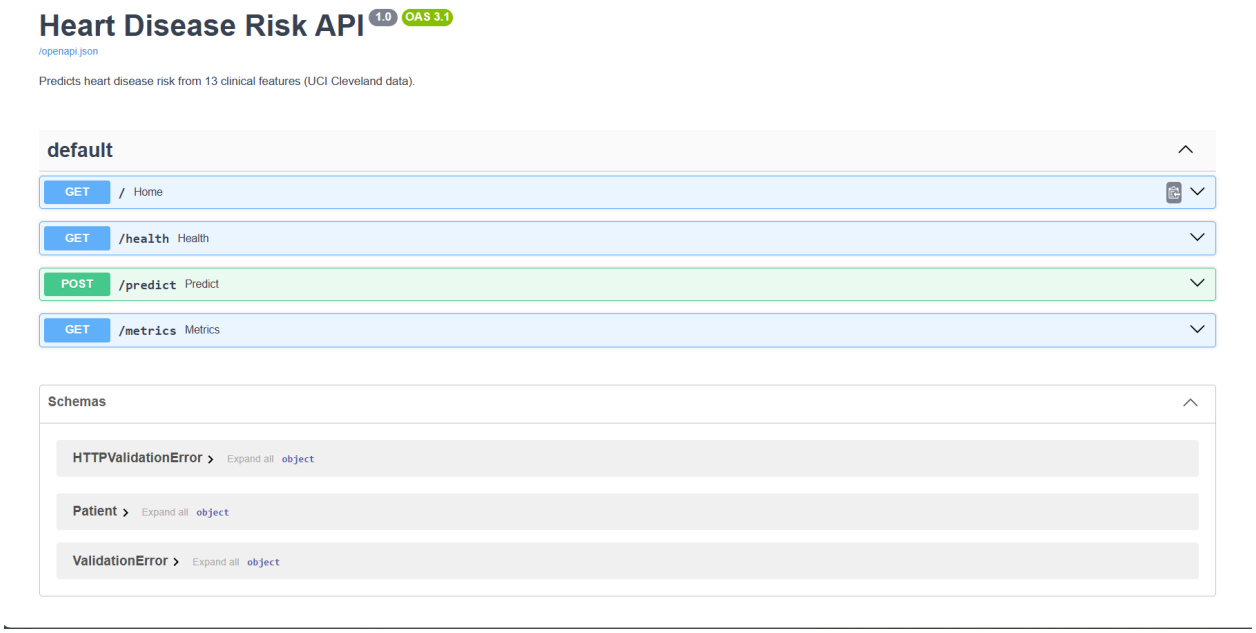



*Figure 19: Swagger UI with a /predict request*

# 10. Production Deployment on Kubernetes

I deployed the Docker image to a local Kubernetes cluster with **Minikube** (Docker driver) inside GitHub Codespaces. The manifests are in `k8s/`:

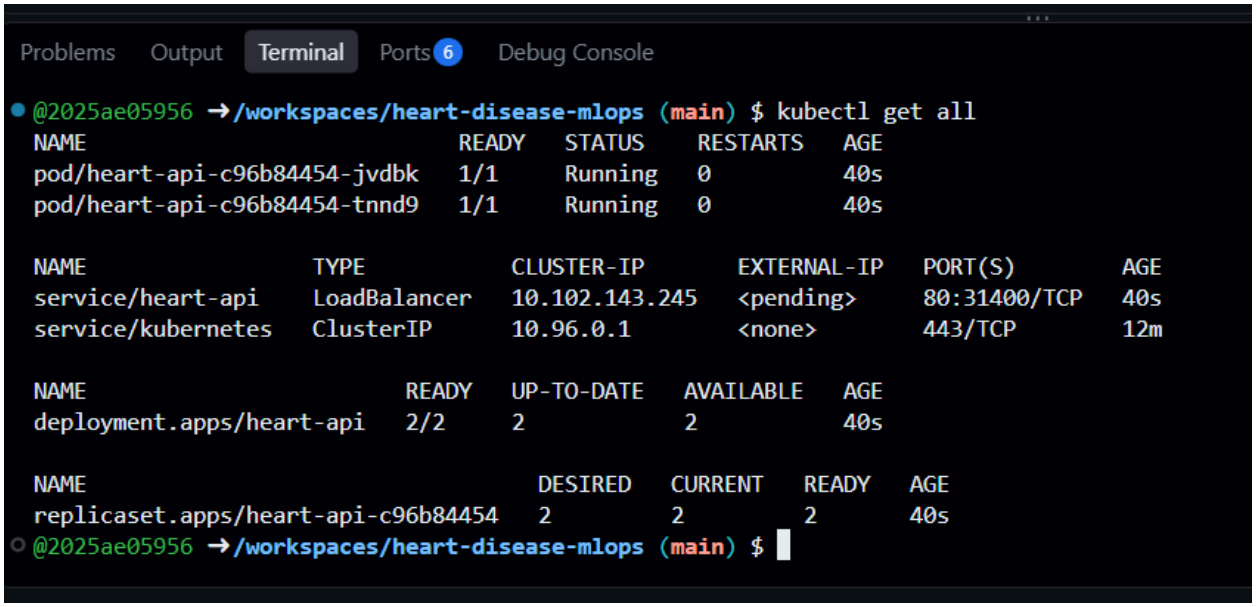
- **`deployment.yaml`:** runs **2 replicas** of the API container, with CPU and memory requests and limits, a **readiness probe** (traffic is only sent to pods whose `/health`

answers) and a **liveness probe** (Kubernetes restarts a container that stops responding). `imagePullPolicy: Never` uses the image loaded into Minikube with `minikube image load`.

- **`service.yaml`:** a Service of type **LoadBalancer** that exposes port 80 and forwards to port 8000 on the pods, sharing requests between the two replicas.
- **`ingress.yaml`:** an NGINX Ingress for the host `heart-api.local` (see below).

Deployment steps:

minikube start --driver=docker --cni=bridge
minikube image load heart-api:latest
kubectl apply -f k8s/deployment.yaml -f k8s/service.yaml
minikube tunnel



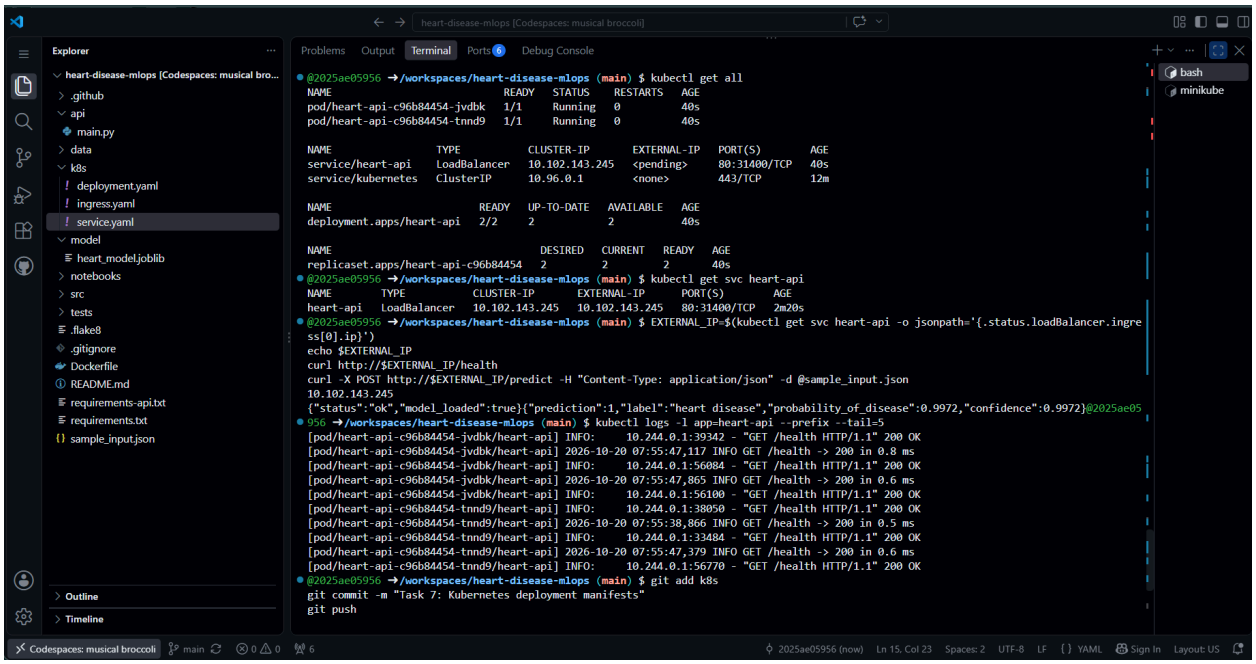



*Figure 20: Pods services and logs*

Both pods reached `1/1 Running` and the deployment showed `2/2` available. `minikube tunnel` assigned the LoadBalancer the external IP **10.102.143.245**. Through this address, `GET /health` returned `model_loaded: true` and `POST /predict` with the sample input returned prediction 1 with confidence 0.9972. The pod logs show requests handled by **both replicas**, confirming the load balancing.

**Ingress:** I also prepared an Ingress manifest, but in the Codespaces environment the Minikube ingress-nginx add-on images could not be downloaded (`ImagePullBackOff`). Since the assignment allows a LoadBalancer **or** an Ingress, I exposed the API through the LoadBalancer service. The same Ingress manifest can be applied unchanged on Docker Desktop or a cloud cluster.

On a public cloud cluster (GKE, EKS or AKS), the same manifests can be used after pushing the image to a container registry and changing the `image:` field; the cloud provider then gives the LoadBalancer a public IP automatically.

---

# 11. Monitoring and Logging

**Logging:** an HTTP middleware in the API logs every request with method, path, status code and response time, and each prediction is logged with its class and probability. The logs are available with `docker logs` and `kubectl logs`.

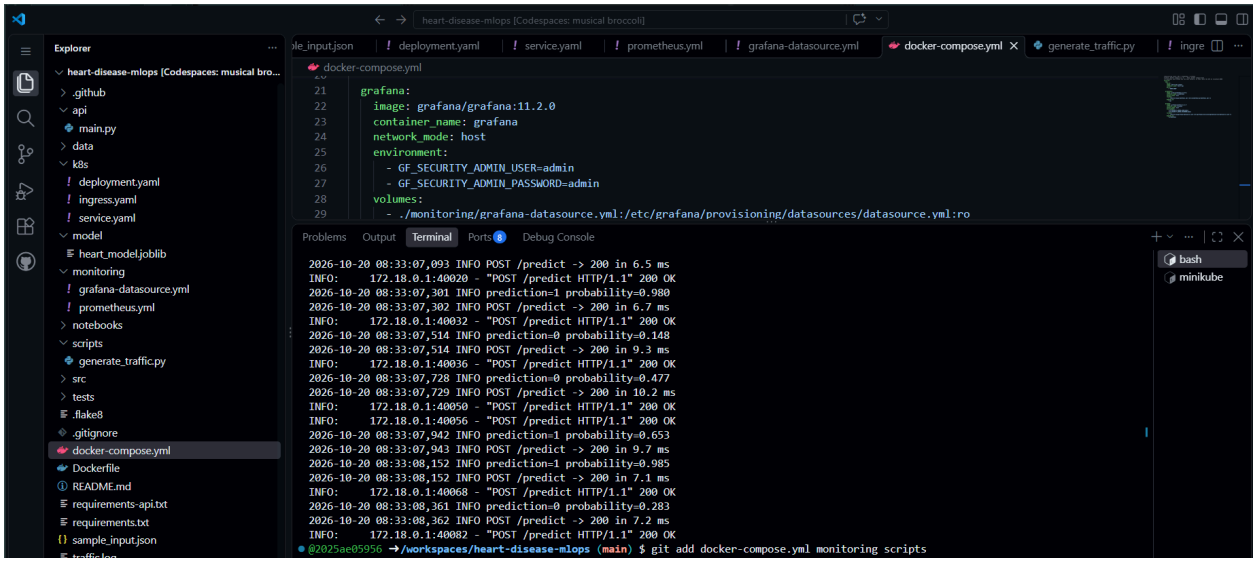
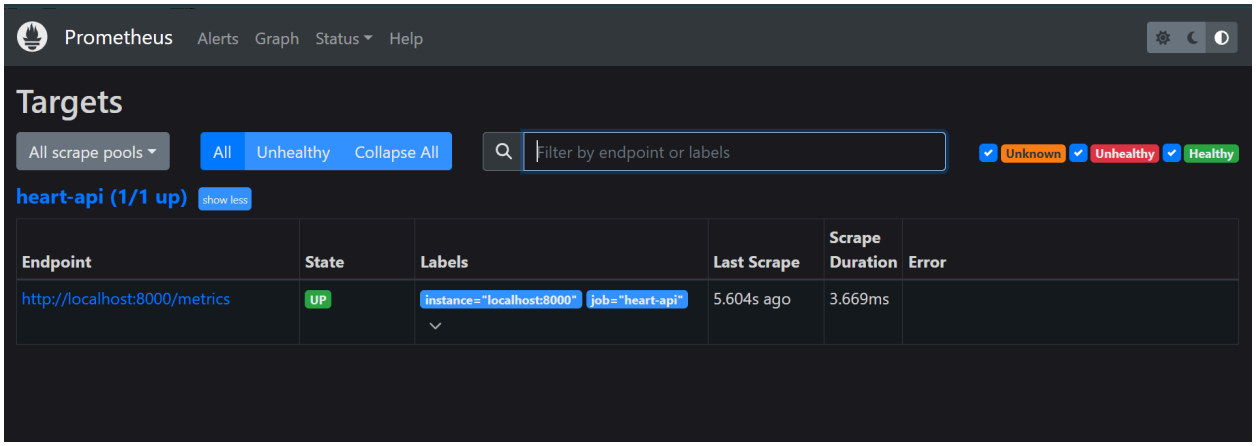


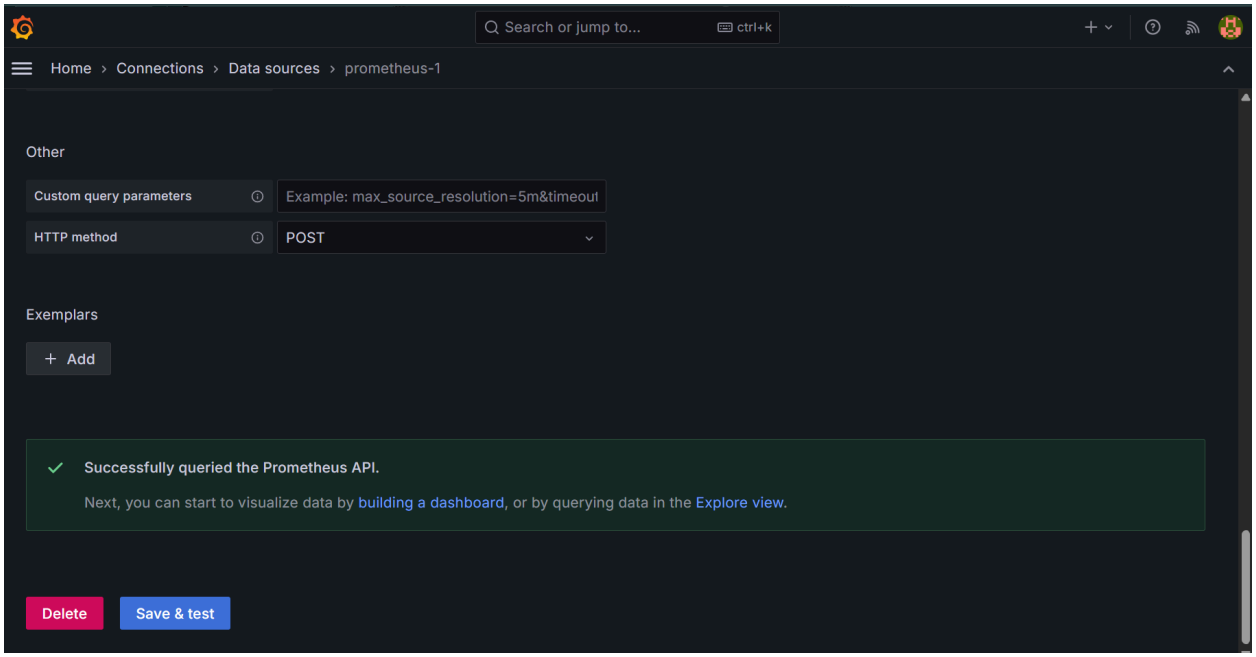

*Figure 21: API request logs*

**Metrics:** the API exposes Prometheus metrics at `/metrics`:

- `api_requests_total`: number of requests by endpoint and status code,
- `api_request_latency_seconds`: histogram of response times,
- `api_predictions_total`: number of predictions per class.

**Monitoring stack:** `docker-compose.yml` starts the API, **Prometheus** (scraping `/metrics` every 5 seconds, configured in `monitoring/prometheus.yml`) and **Grafana** (with Prometheus added as a data source by `monitoring/grafana-datasource.yml`). The Kubernetes pods also carry Prometheus scrape annotations.


Prometheus
Alerts
Graph
Status
Help
Targets
All scrape pools
All
Unhealthy
Collapse All
Filter by endpoint or labels
Unknown
Unhealthy
Healthy
heart-api (1/1 up)
show less
Endpoint
State
Labels
Last Scrape
Scrape Duration
Error
http://localhost:8000/metrics
UP
instance="localhost:8000"
job="heart-api"
5.604s ago
3.669ms


Search or jump to...
ctrl+k
Home
Connections
Data sources
prometheus-1
Other
Custom query parameters
Example: max_source_resolution=5m&timeout
HTTP method
POST
Exemplars
Add
Successfully queried the Prometheus API.
Next, you can start to visualize data by building a dashboard, or by querying data in the Explore view.
Delete
Save & test

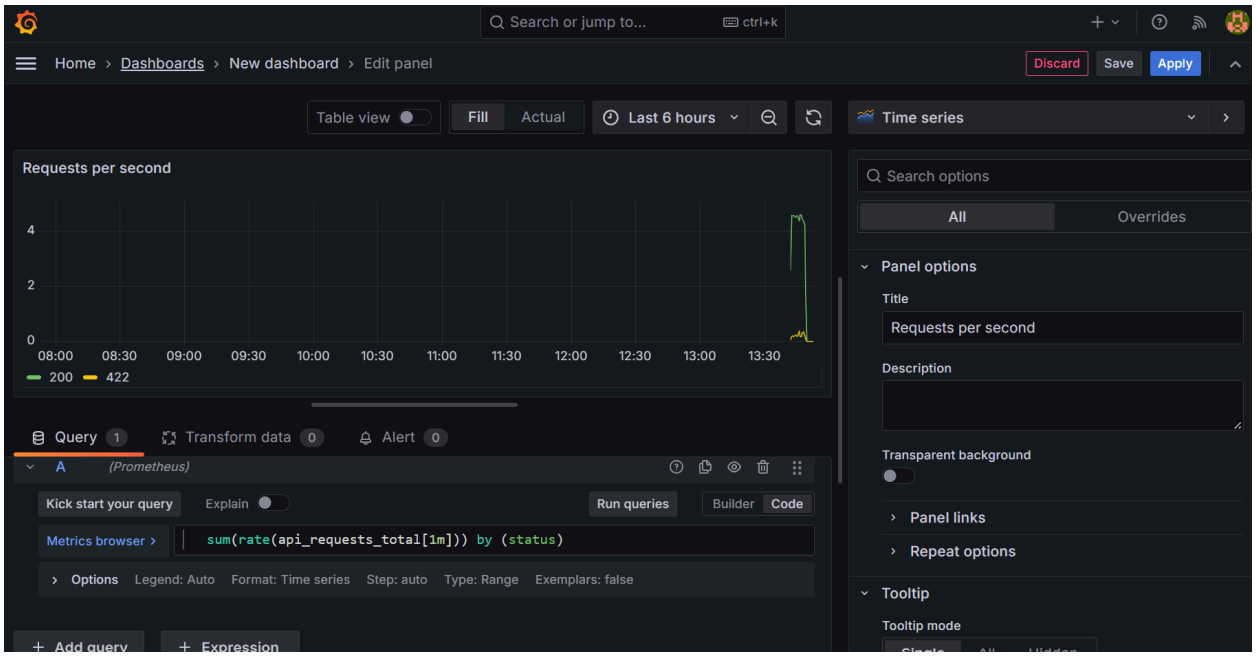



*Figure 22: Prometheus scraping the API (target UP)*

To generate realistic load, `scripts/generate_traffic.py` sends random patient records, about 5% of them deliberately invalid. In the first run of 200 requests, 195 were answered successfully and 5 were rejected with HTTP 422.

I built a Grafana dashboard, **“Heart Disease API Monitoring”**, with four panels:

1. **Requests per second** by status code (traffic),
2. **Predictions by class** (model output; a sudden change in this mix can be an early sign that the incoming data has changed),
3. **Response time (95th percentile)** (speed),
4. **Error rate** (share of invalid requests).

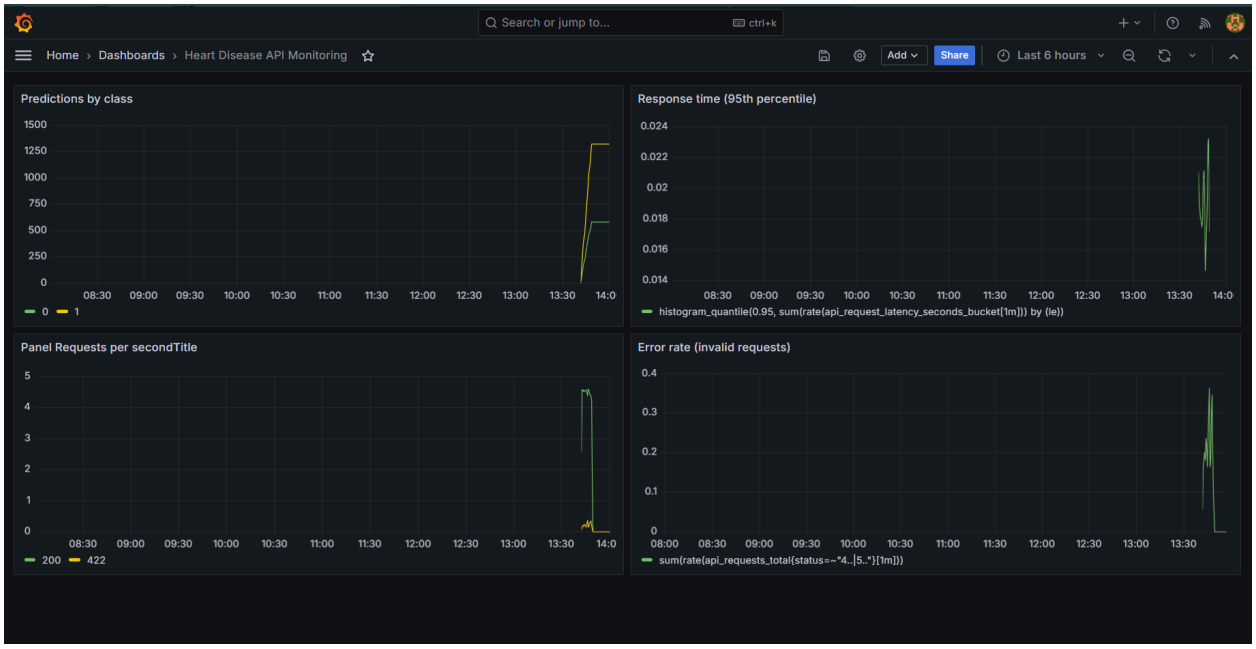



*Figure 23: Grafana monitoring dashboard*

---

# 12. Challenges and Solutions

| Problem | Cause | Solution |
|---|---|---|
| MLflow refused the `./mlruns` folder store | MLflow 3 put the file store in maintenance mode | switched to a SQLite backend (`sqlite:///heart_mlflow.db`) |
| MLflow could not save the models | the secure skops format blocks unreviewed types | listed the scikit-learn column-transformer and tree classes as trusted |
| MLflow UI did not load in Colab | host-header and cross-origin security checks | started the UI with `--allowed-hosts` and `--cors-allowed-origins` |
| First CI run failed at lint | `src/train.py` was only partly pasted | completed the file; the failure showed the pipeline stops on errors |
| `pip install` very slow in Codespaces | the pinned versions had to be compiled for a different Python version | used the model artifact from GitHub Actions; Docker installs its own packages |

| | | |
|---|---|---|
| Minikube node `NotReady` | network plugin not initialised | recreated the cluster with the bridge CNI (`--cni=bridge`) |
| Ingress add-on failed | images could not be pulled (`ImagePullBackOff`) | exposed the API with the LoadBalancer service instead |
| Prometheus target DOWN | Docker's internal DNS was unavailable in Codespaces | ran Prometheus and Grafana with host networking (`localhost:8000`) |

These problems were good practice in reading logs and error messages, which is a large part of real MLOps work.

---

# 13. Production-Readiness

| Requirement | Evidence |
|---|---|
| All scripts execute from a clean setup using the requirements file | every GitHub Actions run starts from a fresh Ubuntu machine, installs only `requirements.txt` and successfully lints, tests and trains (Figure 13) |
| Model serves correctly in an isolated environment | Docker image built and tested with the sample input (Figures 16–17), and the same image served from Kubernetes (Figure 20) |
| Pipeline fails on code or test errors with clear logs | the first run stopped at lint with the exact file, line and error code (Figure 12); chained jobs and `pipefail` stop the pipeline on any error |

---

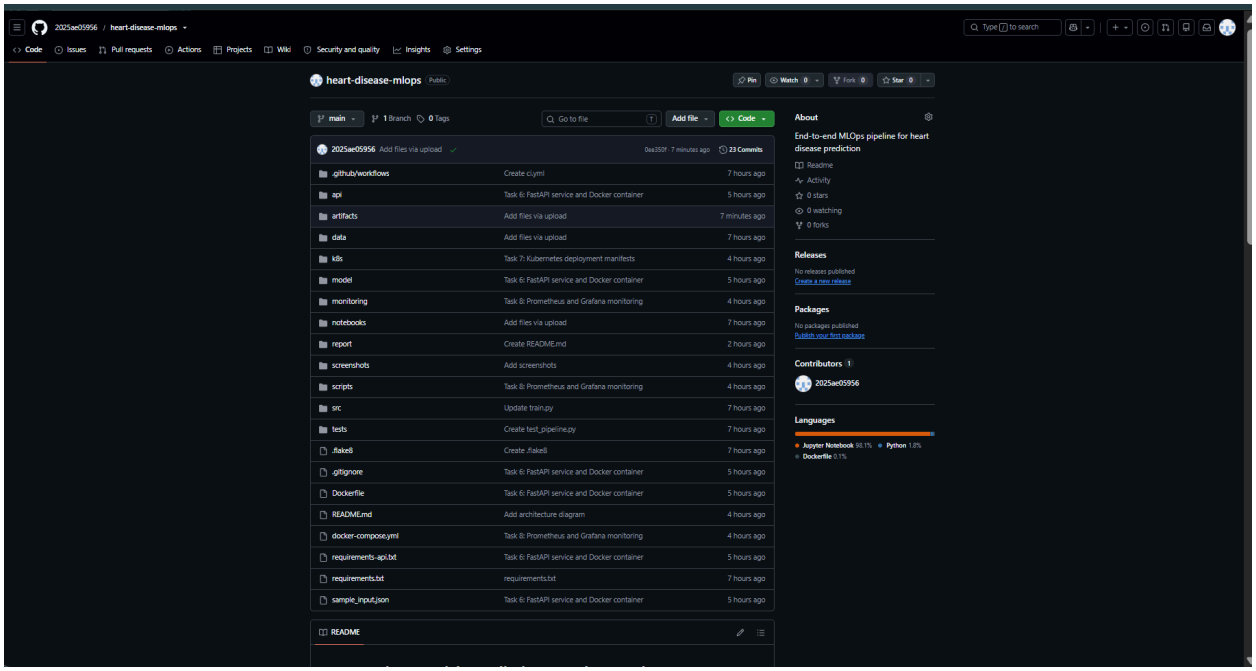
# 14. Deliverables

| Deliverable | Location in the repository |
|---|---|
| Code | `src/`, `api/`, `scripts/` |
| Dockerfile | `Dockerfile` (+ `docker-compose.yml`) |
| Requirements | `requirements.txt`, `requirements-api.txt` |
| Cleaned dataset | `data/heart_clean.csv` (raw file in `data/raw/`) |

| | |
|---|---|
| Download script / instructions | `scripts/download_data.py`, README |
| Notebooks (EDA, training, inference) | `notebooks/01_EDA_Heart_Disease.ipynb`, `02_Model_Training.ipynb`, `03_Model_Packaging.ipynb` (includes inference on a new patient); inference script: `api/main.py` |
| Unit tests | `tests/test_pipeline.py` |
| CI workflow | `.github/workflows/ci.yml` |
| Deployment manifests | `k8s/deployment.yaml`, `k8s/service.yaml`, `k8s/ingress.yaml` |
| Monitoring configuration | `monitoring/` |
| Screenshots | `screenshots/` |
| Trained model | `model/heart_model.joblib` |
| Report | this document |
| API access | not publicly hosted; local access instructions in Section 3 and the README |
| Video | final_video_recordings.mp4 (https://drive.google.com/file/d/19ImrMSql9Ag9qQ2e4SnRYUP759TA1wsL/view?usp=sharing) |

GITHUB:

https://github.com/2025ae05956/heart-disease-mlops



README

# Heart Disease Risk Prediction - End-to-End MLOps

MLOps Assignment 1 (AIMLCZG523). A classifier that predicts heart disease from 13 clinical features (UCI Heart Disease, Cleveland), served as a monitored, containerised API.

## Repository structure

## Architecture



| Path | Contents |
|---|---|
| `scripts/download_data.py` | downloads the UCI data and writes the cleaned CSV |
| `data/heart_clean.csv` | cleaned dataset (303 patients, binary target) |
| `notebooks/` | 01 EDA, 02 model training + MLflow, 03 packaging + inference (Google Colab) |
| `src/heart_pipeline.py` | reusable cleaning + preprocessing pipeline and final model |
| `src/train.py` | training script used by CI (CV, test metrics, MLflow, quality gate) |
| `tests/` | pytest unit tests for data processing and model |
| `.github/workflows/ci.yml` | GitHub Actions: lint -> test -> train, with artifacts |
| `api/main.py`, `Dockerfile` | FastAPI service ( `/predict` , `/health` , `/metrics` ) and container |
| `k8s/` | Kubernetes Deployment, LoadBalancer Service, Ingress |
| `docker-compose.yml` , `monitoring/` | API + Prometheus + Grafana monitoring stack |
| `screenshots/` | screenshots used in the report |

## Setup

```
git clone https://github.com/2025ae05956/heart-disease-mlops.git
cd heart-disease-mlops
python -m venv .venv && source .venv/bin/activate   # Windows: .venv\Scripts\activate
pip install -r requirements.txt                     # Python 3.13
```

README

```
git clone https://github.com/2025ae05956/heart-disease-mlops.git
cd heart-disease-mlops
python -m venv .venv && source .venv/bin/activate   # Windows: .venv\Scripts\activate
pip install -r requirements.txt                     # Python 3.13
```

## Run the pipeline

```
python scripts/download_data.py      # get + clean data
python src/train.py                  # train, evaluate, log to MLflow (outputs/)
flake8 src tests                     # lint
pytest tests -v                      # unit tests
```

Notebooks run in Google Colab: upload the notebook and `data/heart_clean.csv`, then Runtime -> Run all.

## Docker

```
docker build -t heart-api .
docker run -d -p 8000:8000 --name heart-api heart-api
curl -X POST http://localhost:8000/predict -H "Content-Type: application/json" -d @sample_input.json
```

Example response: `{"prediction":1,"label":"heart disease","probability_of_disease":0.9972,"confidence":0.9972}`
Interactive docs: http://localhost:8000/docs

## Kubernetes (Minikube)

```
minikube start --driver=docker --cni=bridge
minikube image load heart-api:latest
kubectl apply -f k8s/deployment.yaml -f k8s/service.yaml
minikube tunnel                      # in a second terminal
kubectl get svc heart-api            # use EXTERNAL-IP
curl http://<EXTERNAL-IP>/health
```

## Monitoring

```
docker compose up -d                 # API :8000, Prometheus :9090, Grafana :3000
python3 scripts/generate_traffic.py --n 200
```

Grafana: http://localhost:3000 -> dashboard "Heart Disease API Monitoring".

## API access

The API is not publicly hosted; run it locally with Docker or Minikube as above.

## Results

Selected model: Logistic Regression (C=1, class_weight=balanced). 5-fold CV ROC-AUC 0.908; test accuracy 0.869, recall 0.929, ROC-AUC 0.958.

---

# 15. Conclusion and Future Work

In this assignment I built a complete MLOps pipeline for heart disease prediction: from raw UCI data, through EDA and a cross-validated, tuned and tracked model, to a tested CI/CD pipeline, a Docker container, a Kubernetes deployment and a monitored API. The selected Logistic Regression detects 93% of the sick patients in the test set (recall 0.929) with a ROC-AUC of 0.958, and the same model is rebuilt identically by the CI pipeline, which shows the process is reproducible.

Possible improvements:

- tune the decision threshold to favour recall even more, since missing a sick patient is costlier than a false alarm;
- add automatic data-drift detection on the logged inputs;
- push the image to a registry and deploy on a public cloud cluster with an Ingress and HTTPS;
- use the MLflow Model Registry to manage model versions and promotion to production;
- add an alerting rule in Grafana for high error rates or slow responses.

---